

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**Phát triển ứng dụng bản đồ số
hỗ trợ sinh viên tìm phòng học
trong khuôn viên Đại học Trà Vinh**

Giảng viên hướng dẫn : **TS THẠCH KỌNG SAOANE**
Sinh viên thực hiện: **MAI TRẦN THANH NHẬT**
Mã số sinh viên: **110122017**
Lớp : **DA22TTA**
Khoá : **2022 - 2026**

Vĩnh Long, tháng 6 năm 2026

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**Phát triển ứng dụng bản đồ số
hỗ trợ sinh viên tìm phòng học
trong khuôn viên Đại học Trà Vinh**

Giảng viên hướng dẫn : **TS THẠCH KỌNG SAOANE**
Sinh viên thực hiện: **MAI TRẦN THANH NHẬT**
Mã số sinh viên: **110122017**
Lớp : **DA22TTA**
Khoá : **2022 - 2026**

Vĩnh Long, tháng 6 năm 2026

LỜI MỞ ĐẦU

Trong thời đại cách mạng công nghiệp 4.0 và chuyển đổi số đang diễn ra mạnh mẽ, việc ứng dụng công nghệ thông tin vào quản lý và hỗ trợ các hoạt động trong môi trường giáo dục đại học đã trở thành một xu hướng tất yếu. Các trường đại học hiện nay không chỉ chú trọng nâng cao chất lượng giảng dạy mà còn đặc biệt quan tâm đến việc tối ưu hóa trải nghiệm không gian học đường, giúp sinh viên tiếp cận các dịch vụ tiện ích một cách nhanh chóng và hiệu quả thông qua các giải pháp công nghệ số hiện đại.

Trong bối cảnh đó, hệ thống thông tin địa lý và công nghệ bản đồ số nổi lên như một giải pháp đột phá trong việc số hóa, không gian hóa thông tin trực quan. Đối với một cơ sở giáo dục có khuôn viên rộng lớn và hệ thống nhiều tòa nhà, khu chức năng như Đại học Trà Vinh, việc phát triển một công cụ hỗ trợ tìm kiếm và định vị phòng học là vô cùng cần thiết. Nó không chỉ giải quyết triệt để những khó khăn của tân sinh viên hay khách tham quan khi mới tiếp cận môi trường học đường, mà còn tối ưu hóa thời gian di chuyển giữa các tiết học.

Đề tài "Phát triển ứng dụng bản đồ số hỗ trợ sinh viên tìm phòng học trong khuôn viên Đại học Trà Vinh" được thực hiện với mục tiêu ứng dụng các công nghệ lập trình web hiện đại kết hợp dữ liệu không gian trực quan nhằm xây dựng một hệ thống bản đồ nội khu thân thiện và chính xác. Ứng dụng hướng tới việc tích hợp các chức năng tìm kiếm phòng học, hiển thị thông tin chi tiết các khu vực và gợi ý lộ trình di chuyển tối ưu cho người dùng trong khuôn viên nhà trường.

Với tinh thần đó, nghiên cứu này không chỉ mang tính ứng dụng thực tiễn cao, giải quyết trực tiếp nhu cầu di chuyển hàng ngày của sinh viên, mà còn hướng tới việc góp phần thúc đẩy lộ trình chuyển đổi số toàn diện tại Đại học Trà Vinh.

LỜI CẢM ƠN

Tôi xin gửi lời cảm ơn chân thành đến quý thầy cô bộ môn Công nghệ thông tin. Thầy cô đã nhiệt tình hỗ trợ tôi trong quá trình làm đồ án, cũng như đóng góp ý kiến để đồ án được hoàn thành.

Đặc biệt, tôi xin gửi lời cảm ơn đến thầy Thạch Kọng Saoane, người đã trực tiếp hướng dẫn, chỉ bảo và hỗ trợ tôi trong quá trình thực hiện đồ án “Phát triển ứng dụng bản đồ số hỗ trợ sinh viên tìm phòng học trong khuôn viên Đại học Trà Vinh”. Thầy đã dành nhiều thời gian, công sức và tâm huyết để giúp đỡ tôi hoàn thành đồ án. Thầy luôn tận tình giải đáp mọi thắc mắc, góp ý chỉnh sửa các sai sót và đưa ra những góp ý quý báu để tôi hoàn thành đồ án một cách tốt nhất.

Trong quá trình thực hiện đồ án, tôi cũng nhận được sự hỗ trợ nhiệt tình của quý thầy cô trong bộ môn. Nhờ sự hướng dẫn và hỗ trợ của quý thầy cô, chúng em đã có cơ hội tiếp cận, học hỏi và hoàn thiện bản thân, không chỉ về chuyên môn mà còn trong cách tư duy và giải quyết vấn đề.

Tôi xin chân thành cảm ơn quý thầy cô đã tạo điều kiện thuận lợi cho tôi trong quá trình thực hiện đồ án.

Vĩnh Long, ngày tháng năm 20...

Sinh viên thực hiện

NHẬN XÉT

[illegible]

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

Họ và tên sinh viên: MSSV:
 Ngành: Khóa:
 Tên đề tài:

Họ và tên Giáo viên hướng dẫn:.....
 Chức danh: Học vị:

1. Nội dung đề tài:

2. Ưu điểm:

3. Khuyết điểm:

4. Điểm mới đề tài:

.....
5. Giá trị thực trên đề tài:

.....
.....
.....
.....
.....
.....
.....

6. Đề nghị sửa chữa bổ sung:

.....
.....
.....
.....
.....
.....
.....

7. Đánh giá:

.....
.....
.....
.....

Vĩnh Long, ngày tháng năm 20...
Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)

NHẬN XÉT

[illegible]

Giảng viên chấm
(ký và ghi rõ họ tên)

(Của cán bộ chấm đề án, khóa luận)

Họ và tên người nhận xét:
 Chức danh: Học vị:
 Chuyên ngành:
 Cơ quan công tác:
 Tên sinh viên:
 Tên đề tài đồ án, khóa luận tốt nghiệp:

I. Ý KIẾN NHÂN XÉT

1. Nội dung:

2. Điểm mới các kết quả của đồ án, khóa luận:

3. Ứng dụng thực tế:

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

MỤC LỤC

CHƯƠNG 1 : ĐẶT VẤN ĐỀ.....	1
1.1 . Lý do chọn đề tài.....	1
1.2 . Mục tiêu	1
1.3 . Nội dung.....	2
1.4 . Đối tượng và phạm vi nghiên cứu.....	2
1.5 . Phương pháp nghiên cứu.....	3
CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT	4
2.1 . Thuật toán A*	4
2.1.1 . Giới thiệu về thuật toán A*	4
2.1.2 . Sự khác nhau giữa thuật toán A* và Dijkstra	4
2.1.3 . Ứng dụng thực tế của thuật toán A*	7
2.2 . Vue	7
2.3 . Nodejs	8
2.4 . PostgreSQL	8
2.4.1 . Hệ quản trị cơ sở dữ liệu PostgreSQL	8
2.4.2 . Tiện ích mở rộng PostGIS	8
2.4.3 . Tiện ích mở rộng pgRouting.....	11
2.5 . Mô hình MVC	11
2.5.1 . Các thành phần trong mô hình MVC.....	11
2.5.2 . Luồng xử lý trong MVC	12
2.5.3 . Ưu và Nhược điểm của mô hình kiến trúc MVC	12
2.6 . Docker	14
2.6.1 . Cơ bản về Docker	14

2.6.2 . Ưu, nhược điểm	14
CHƯƠNG 3 : HIỆN THỰC HÓA NGHIÊN CỨU	16
3.1 . Thiết kế cơ sở dữ liệu.....	16
3.1.1 . Lược đồ use case tổng quan của hệ thống	16
3.1.2 . Mô tả các use case	17
3.1.3 . Mô hình dữ liệu ERD.....	20
3.1.4 . Chi tiết các thực thể	21
3.2 . Xây dựng Backend REST API.....	25
3.2.1 . Xây dựng Backend theo mô hình MVC	25
3.2.2 . Xây dựng REST API	27
3.3 . Phát triển giao diện và thuật toán.....	30
3.3.1 . Kiến trúc ứng dụng SPA với Vue 3 Composition API.....	30
3.3.2 . Cơ chế tích hợp nền bản đồ qua Leaflet	30
3.3.3 . Thuật toán tìm đường.....	31
CHƯƠNG 4 : KẾT QUẢ NGHIÊN CỨU	36
4.1 . Giao diện trang chủ	36
4.2 . Giao diện trang bản đồ	36
4.3 . Giao diện trang tin tức.....	37
4.4 . Giao diện trang danh sách tòa nhà	38
4.5 . Giao diện trang quản trị	39
CHƯƠNG 5 : KẾT LUẬN VÀ PHƯƠNG HƯỚNG PHÁT TRIỂN	40
5.1 . Kết luận	40
5.2 . Hướng phát triển	40
DANH MỤC TÀI LIỆU THAM KHẢO	41

DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

Hình 3.1 Mô hình use case tổng quan của hệ thống.....	16
Hình 3.2 Mô hình use case người dùng sử dụng bản đồ	17
Hình 3.3 Mô hình use case xem bài đăng.....	18
Hình 3.4 Mô hình use case quản trị viên	19
Hình 3.5 Mô hình cơ sở dữ liệu ERD.....	20
Hình 3.6 Sơ đồ luồng xử lý yêu cầu HTTP tại Backend.....	26
Hình 3.7 Phương thức thiết lập cấu hình bản đồ tương tác Leaflet	30
Hình 3.8 Định nghĩa tọa độ giới hạn ranh giới khuôn viên ĐH Trà Vinh	32
Hình 3.9 Phương thức lấy tọa độ vị trí hiện tại của người dùng	33
Hình 3.10 Thuật toán tìm road_node gần nhất với tọa độ (lng, lat).....	33
Hình 3.11 Thuật toán xác định và lựa chọn cổng tiếp cận khuôn viên trường ..	34
Hình 3.12 Module tìm đường OSRM.....	34
Hình 3.13 Sơ đồ luồng tiến trình xử lý tìm đường	35
Hình 4.1 Giao diện trang chủ	36
Hình 4.2 Giao diện trang bản đồ	36
Hình 4.3 Giao diện chức năng tìm kiếm trong trang bản đồ	37
Hình 4.4 Giao diện trang tin tức.....	37
Hình 4.5 Giao diện trang đọc tin tức	38
Hình 4.6 Giao diện trang danh sách tòa nhà.....	38
Hình 4.7 Giao diện trang quản trị.....	39

Bảng 2.1: So sánh thuật toán Dijkstra và thuật toán A*	7
Bảng 3.1 Bảng thực thể road_nodes	21
Bảng 3.2 Bảng thực thể roads	22
Bảng 3.3 Bảng thực thể landmarks	23
Bảng 3.4 Bảng thực thể rooms	24
Bảng 3.5 Bảng thực thể news	25
Bảng 3.6 Bảng thực thể admin_config	25
Bảng 3.7 Danh mục các API Landmark	28
Bảng 3.8 Danh mục các API Rooms	28
Bảng 3.9 Danh mục các API Paths	29
Bảng 3.10 Danh mục các API News	29

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, Durability
CMS	Content Management System
CRUD	Create, Read, Update, Delete
ERD	Entity-Relationship Diagram
GIS	Geographic Information System
GiST	Generalized Search Tree
ISO	International Organization for Standardization
OSRM	Open Source Routing Machine
SoC	Separation of Concerns
SRID	Spatial Reference System Identifier
WGS 84	World Geodetic System 1984
UTM	Universal Transverse Mercator

CHƯƠNG 1: ĐẶT VẤN ĐỀ

1.1. Lý do chọn đề tài

Trong những năm gần đây, xu hướng chuyển đổi số trong giáo dục đại học đã và đang trở thành một yêu cầu cấp thiết nhằm tối ưu hóa công tác quản lý và nâng cao trải nghiệm học tập của sinh viên. Đại học Trà Vinh với quy mô khuôn viên rộng lớn bao gồm nhiều khối nhà học, phòng thực hành, trung tâm nghiên cứu và các khu vực tiện ích (như thư viện, nhà xe, khu thể thao), mang lại một môi trường học tập tiện nghi nhưng cũng gây ra không ít trở ngại trong việc định vị không gian. Đối với tân sinh viên, học viên các khóa mới cũng như khách tham quan, việc tìm kiếm chính xác vị trí phòng học hoặc các tòa nhà trong những ngày đầu đến trường luôn là một thách thức lớn, dẫn đến việc mất thời gian và ảnh hưởng đến hiệu suất học tập, làm việc.

Mặc dù sơ đồ tổng thể của nhà trường đã được công bố, song các bản vẽ tĩnh dạng 2D truyền thống hoặc hình ảnh mô phỏng hiện tại còn bộc lộ nhiều hạn chế: thiếu tính tương tác trực quan, không hỗ trợ chức năng tìm kiếm động và đặc biệt là không thể chỉ đường theo thời gian thực. Trong khi đó, các công nghệ bản đồ phổ biến như Google Maps thường chỉ hỗ trợ định vị các tuyến đường giao thông lớn bên ngoài, chưa số hóa sâu và chi tiết hệ thống giao thông nội khu cũng như sơ đồ từng phòng học của các tòa nhà trong khuôn viên trường.

Xuất phát từ thực tế đó, việc xây dựng một hệ thống bản đồ số chuyên biệt, có khả năng tích hợp không gian địa lý nội khu cùng các thuật toán tìm đường tối ưu là vô cùng cần thiết. Ứng dụng không chỉ giúp sinh viên tra cứu vị trí nhanh chóng trên các thiết bị di động và máy tính, mà còn cung cấp một công cụ quản lý trực quan cho nhà trường trong việc cập nhật thông tin hạ tầng. Vì các lý do trên, tôi đã quyết định lựa chọn thực hiện đề tài: "Phát triển ứng dụng bản đồ số hỗ trợ sinh viên tìm phòng học trong khuôn viên Đại học Trà Vinh".

1.2. Mục tiêu

Số hóa toàn bộ sơ đồ các tòa nhà, phòng học và hạ tầng giao thông nội khu Đại học Trà Vinh. Cho phép sinh viên tra cứu vị trí phòng học, tòa nhà và các tiện ích (thư viện, nhà xe,...) trên nền tảng WebApp. Giải quyết vấn đề khó khăn trong việc tìm kiếm phòng học của sinh viên mới và khách tham quan tại trường.

1.3. Nội dung

- Khảo sát và thu thập dữ liệu: Thu thập tọa độ GPS, sơ đồ mặt bằng các tầng và thông tin định danh phòng học tại các khu vực thuộc Đại học Trà Vinh.
- Thiết kế hệ thống:
 - Backend: Sử dụng Node.js để xây dựng các API xử lý truy vấn không gian và logic tìm đường.
 - Frontend: Xây dựng giao diện người dùng bằng Vue.js, đảm bảo tính phản hồi trên thiết bị di động.
 - Database: Thiết kế cơ sở dữ liệu quan hệ với PostgreSQL và phần mở rộng PostGIS để quản lý dữ liệu địa lý.
- Phát triển tính năng:
 - Tích hợp bản đồ Leaflet và lớp dữ liệu GeoJSON.
 - Xây dựng module tìm đường sử dụng thuật toán A^* để tính toán lộ trình tối ưu.
 - Chức năng tìm kiếm nhanh phòng học theo mã hoặc tên phòng.
- Kiểm thử và triển khai: Đánh giá độ chính xác của chỉ đường và hiệu năng của ứng dụng trên thực tế.

1.4. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Các mô hình dữ liệu không gian địa lý và phương pháp số hóa bản đồ nội khu.
- Thuật toán tìm đường ngắn nhất A^* ứng dụng trong mạng lưới giao thông đường bộ.
- Kiến trúc phát triển ứng dụng Web Full-stack.

Phạm vi nghiên cứu:

- Phạm vi không gian: Giới hạn trong khuôn viên khu I của Trường Đại học Trà Vinh.
- Phạm vi chức năng: Số hóa đến cấp độ tòa nhà, danh mục phòng học thuộc tòa nhà, các trục đường nội bộ dành cho người đi bộ và xe máy; chưa đi sâu vào sơ

đồ kiến trúc chi tiết bên trong từng tầng dạng 3D (chỉ hỗ trợ danh sách thuộc tính phòng theo tòa nhà).

1.5. Phương pháp nghiên cứu

Phương pháp nghiên cứu lý thuyết: Nghiên cứu các tài liệu khoa học, sách giáo trình và tài liệu kỹ thuật chính thức về GIS, PostGIS, pgRouting, thuật toán đồ thị và các framework lập trình hiện đại nhằm xây dựng cơ sở lý luận vững chắc cho đề tài.

Phương pháp thực nghiệm, khảo sát thực địa: Trực tiếp khảo sát khuôn viên Trường Đại học Trà Vinh để xác định tọa độ các mốc giải tích, vẽ lại sơ đồ các tuyến đường nội khu, thu thập chính xác tên gọi, chức năng của từng phòng học và tòa nhà để làm phôi dữ liệu đầu vào.

Phương pháp phân tích và thiết kế hệ thống: Áp dụng quy trình kỹ thuật phân mềm để phân tích yêu cầu người dùng, thiết kế sơ đồ luồng dữ liệu, sơ đồ quan hệ thực thể (ERD) và kiến trúc phân tầng hệ thống.

Phương pháp chuyên gia và thử nghiệm thực tế: Triển khai xây dựng phiên bản thử nghiệm, tiến hành chạy thử các kịch bản tìm đường từ các điểm nút khác nhau trong trường, đo lường tốc độ phản hồi của API và độ trễ của WebSocket, từ đó tham khảo ý kiến phản hồi của người dùng để tối ưu hóa giao diện và hiệu năng ứng dụng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Thuật toán A*

2.1.1. Giới thiệu về thuật toán A*

Thuật toán A* là một trong những thuật toán tìm đường đi phát triển từ thuật toán Dijkstra, được sử dụng rộng rãi trong khoa học máy tính và trí tuệ nhân tạo nhờ vào tính hiệu quả và tối ưu của nó. A* là thuật toán tìm kiếm dựa trên đồ thị, sử dụng thông tin heuristic (hàm ước lượng) để định hướng tìm kiếm, giúp giảm thiểu số lượng nút cần duyệt qua so với các thuật toán tìm kiếm mù (blind search). Nguyên lý hoạt động của A* dựa trên việc tối thiểu hóa hàm tổng chi phí đánh giá tại mỗi nút n :

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: Chi phí thực tế đi từ điểm xuất phát ban đầu đến nút hiện tại n .
- $h(n)$: Chi phí ước lượng (heuristic) từ nút hiện tại n đến điểm đích. Hàm $h(n)$ đóng vai trò quyết định độ hiệu quả của thuật toán. Trong bài toán không gian địa lý, $h(n)$ thông thường được tính bằng khoảng cách đường chim bay (khoảng cách Euclidean hoặc Manhattan) giữa nút n và đích.
- $f(n)$: Tổng chi phí ước lượng thấp nhất của đường đi qua nút.

2.1.2. Sự khác nhau giữa thuật toán A* và Dijkstra

Để hiểu rõ sự khác biệt giữa hai thuật toán định tuyến kinh điển này, cần phân tích từ bản chất toán học của hàm chi phí, cơ chế mở rộng không gian tìm kiếm, cho đến hiệu năng thực tế khi triển khai trên mạng lưới đồ thị giao thông.

2.1.2.1. Bản chất toán học và Hàm chi phí (Cost Functions)

Thuật toán Dijkstra là một thuật toán tìm kiếm mù hoặc tìm kiếm không có thông tin (Uninformed Search). Hàm chi phí tại một nút n bất kỳ chỉ phụ thuộc vào quá khứ:

$$f(n) = g(n)$$

Trong đó, $g(n)$ là tổng chi phí thực tế (khoảng cách, thời gian hoặc độ trễ) tích lũy từ nút gốc đến nút n . Do đó, Dijkstra chỉ quan tâm đến việc mở rộng đường đi ngắn nhất hiện có từ điểm xuất phát mà không hề có định hướng về vị trí của điểm đích.

Ngược lại, thuật toán A^* là thuật toán tìm kiếm có thông tin hay tìm kiếm heuristic (Informed Search). Nó bổ sung một thành phần dự báo tương lai thông qua hàm heuristic $h(n)$:

$$f(n) = g(n) + h(n)$$

Trong đó, $h(n)$ là chi phí ước lượng từ nút hiện tại n đến điểm đích. Thành phần này đóng vai trò như một "la bàn" định hướng, hướng luồng tính toán tập trung về phía mục tiêu, giúp giảm thiểu đáng kể số lượng nút phải duyệt qua.

Hệ quả toán học: Thuật toán Dijkstra chính là một trường hợp đặc biệt của thuật toán A^* khi hàm toán học $h(n) = 0$ với mọi nút n trên đồ thị.

2.1.2.2. Cơ chế mở rộng không gian tìm kiếm

Sự khác biệt lớn nhất về mặt trực quan giữa hai thuật toán nằm ở hình dạng không gian tìm kiếm khi lan truyền trên đồ thị không gian phẳng:

- **Thuật toán Dijkstra:** Hoạt động theo cơ chế loang đều ra mọi hướng theo dạng sóng tròn đồng tâm (hoặc hình elip tùy thuộc vào trọng số cạnh). Thuật toán sẽ quét qua tất cả các nút có chi phí thấp hơn khoảng cách đến đích, bất kể nút đó nằm cùng hướng hay ngược hướng với điểm đích. Điều này dẫn đến việc lãng phí tài nguyên tính toán vào những khu vực không liên quan.
- **Thuật toán A^* :** Nhờ có hàm $h(n)$ thường được tính bằng khoảng cách Euclid đường chim bay hoặc khoảng cách Manhattan trên bản đồ số, không gian tìm kiếm bị bóp nghẹt và kéo dài theo một trục hướng thẳng về phía đích. Thuật toán ưu tiên mở rộng các nút nằm trên đường đi ngắn nhất tiềm năng, tạo ra một dải không gian tìm kiếm hẹp và có mục tiêu rõ rệt.

2.1.2.3. Điều kiện tối ưu và Hàm Heuristic chấp nhận được

- **Dijkstra:** Luôn luôn đảm bảo tìm được đường đi ngắn nhất tuyệt đối trên đồ thị, với điều kiện duy nhất là trọng số của tất cả các cạnh phải không âm ($\text{cost} \geq 0$).

- A^* : Tính tối ưu của A^* phụ thuộc hoàn toàn vào đặc tính của hàm ước lượng $h(n)$. A^* chỉ đảm bảo tìm được đường đi ngắn nhất chính xác nếu $h(n)$ là một hàm chấp nhận được, nghĩa là:

$$0 \leq h(n) \leq h^*(n), \quad \forall n$$

Trong đó $h^*(n)$ là chi phí thực tế tối ưu từ n đến đích. Nếu hàm $h(n)$ đánh giá quá cao chi phí thực tế ($h(n) > h^*(n)$), thuật toán A^* có thể chạy nhanh hơn nhưng sẽ bỏ qua đường đi ngắn nhất và trả về một kết quả cận tối ưu. Ngoài ra, trong đồ thị không gian liên tục, $h(n)$ còn cần thỏa mãn tính nhất quán để đảm bảo một khi nút đã được duyệt qua thì chi phí tìm đến nó đã là tối ưu nhất.

2.1.2.4. Bảng so sánh tổng hợp hiệu năng

Tiêu chí so sánh	Thuật toán Dijkstra	Thuật toán A^*
Phân loại thuật toán	Tìm kiếm không có thông tin (Uninformed Search).	Tìm kiếm có hướng dẫn/thông tin (Informed Search).
Hàm đánh giá $f(n)$	$f(n) = g(n)$ (Chỉ tính chi phí đã đi qua).	$f(n) = g(n) + h(n)$ (Kết hợp chi phí thực tế và ước lượng).
Hướng tìm kiếm	Đa hướng, loang đều như làn sóng vòng tròn từ tâm gốc.	Đơn hướng, tập trung kéo dài về phía điểm đích.
Độ phức tạp thời gian	Thường là $O(E + V \log V)$ với Heap dữ liệu. Phải duyệt phần lớn đồ thị.	Phụ thuộc vào hàm heuristic. Trong trường hợp tốt nhất có thể đạt $O(V)$, trường hợp xấu nhất $h(n)$ bằng Dijkstra.
Hiệu suất bộ nhớ	Tốn bộ nhớ hơn do phải lưu trữ nhiều trạng thái nút không cần thiết vào Open Set.	Tiết kiệm bộ nhớ hơn nhờ thu hẹp tối đa số lượng nút cần quản lý trong hàng đợi ưu tiên.

Trường hợp sử dụng lý tưởng	Khi cần tìm đường đi ngắn nhất từ một điểm đến tất cả các điểm còn lại trên bản đồ.	Khi chỉ cần tìm đường đi tối ưu giữa một cặp điểm xuất phát và đích xác định.
-----------------------------	---	---

Bảng 2.1: So sánh thuật toán Dijkstra và thuật toán A*

2.1.3. Ứng dụng thực tế của thuật toán A*

Nhờ vào sự cân bằng hoàn hảo giữa tốc độ tính toán và tính tối ưu, A* được ứng dụng rộng rãi trong:

- **Hệ thống thông tin địa lý và Bản đồ số:** Tìm đường đi tối ưu trên mạng lưới giao thông đường bộ (áp dụng qua các thư viện mở rộng như pgRouting trong PostgreSQL bằng hàm pgr_astar).
- **Trí tuệ nhân tạo trong Video Games:** Định tuyến di chuyển cho các NPC vượt qua các chướng ngại vật phức tạp trong thời gian thực.
- **Robot tự hành:** Hỗ trợ robot thiết lập quỹ đạo di chuyển trong môi trường nhà xưởng hoặc bản đồ đã được số hóa.

2.2. Vue

Vue là một framework lũy tiến dựa trên ngôn ngữ JavaScript dùng để xây dựng giao diện người dùng mã nguồn mở.

- **Kiến trúc cốt lõi:** Khác với các framework nguyên khối khác, Vue được thiết kế từ đầu theo hướng có thể áp dụng dần dần. Thư viện lõi chỉ tập trung vào lớp giao diện.
- **Composition API:** Được giới thiệu mạnh mẽ trong Vue 3, mang lại khả năng tổ chức mã nguồn linh hoạt hơn, cho phép gom nhóm các logic liên quan lại với nhau thay vì phân tách theo Options API truyền thống (data, methods, computed). Điều này tăng tính tái sử dụng mã và tối ưu hóa hiệu năng gộp bundle.
- **Virtual DOM & Phản xạ:** Vue sử dụng cơ chế phản xạ dựa trên ES6 Proxy, theo dõi các thay đổi của trạng thái dữ liệu một cách tự động và cập nhật một cách tối ưu lên Virtual DOM, giúp render giao diện siêu nhanh mà không cần tải lại toàn bộ trang.

2.3. Nodejs

Node.js không phải là một framework mà là một môi trường thực thi mã nguồn mở, đa nền tảng, cho phép lập trình viên chạy mã JavaScript trực tiếp trên phía máy chủ

- **V8 Engine:** Node.js được xây dựng trên nền tảng bộ thực thi JavaScript V8 của Google Chrome, giúp biên dịch trực tiếp mã JavaScript sang mã máy bản địa với hiệu năng cực cao.
- **Kiến trúc hướng sự kiện, phi bất đồng bộ (Event-driven, Non-blocking I/O):** Đây là đặc trưng quan trọng nhất của Node.js. Thay vì tạo ra một luồng mới cho mỗi yêu cầu kết nối như các kiến trúc truyền thống (Thread-per-request của Apache/PHP), Node.js hoạt động đơn luồng kết hợp với cơ chế Event Loop. Khi gặp các tác vụ tốn thời gian (như truy vấn cơ sở dữ liệu Spatial PostGIS hoặc đọc ghi file), Node.js sẽ đẩy tác vụ đó ra nền tảng bất đồng bộ và tiếp tục nhận các request khác.
- **Phù hợp cho ứng dụng thời gian thực:** Nhờ cơ chế Non-blocking, Node.js xử lý cực tốt hàng vạn kết nối đồng thời với tài nguyên phần cứng tối thiểu, là nền tảng hoàn hảo để triển khai các công giao tiếp WebSocket phục vụ truyền tải luồng thông báo thời gian thực giữa Admin và các Client.

2.4. PostgreSQL

2.4.1. Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ-đối tượng mã nguồn mở mạnh mẽ, nổi tiếng về độ tin cậy, tính toàn vẹn dữ liệu cực cao và khả năng mở rộng không giới hạn, hỗ trợ hoàn toàn các chuẩn giao dịch ACID.

2.4.2. Tiện ích mở rộng PostGIS

2.4.2.1. Giới thiệu chung và Kiến trúc Không gian

PostGIS là một tiện ích mở rộng mã nguồn mở dành cho hệ quản trị cơ sở dữ liệu quan hệ đối tượng PostgreSQL, đóng vai trò biến cơ sở dữ liệu này thành một Cơ sở dữ liệu không gian địa lý. PostGIS tuân thủ chặt chẽ đặc tả *Simple Features for SQL* được

đặt ra bởi Tổ chức Không gian Địa lý Mở (OGC) và Tổ chức Tiêu chuẩn hóa Quốc tế (ISO 19125).

Kiến trúc của PostGIS cho phép lưu trữ các dữ liệu có tọa độ địa lý trực tiếp trong các bảng của PostgreSQL dưới hai dạng kiểu dữ liệu chính:

- **Geometry:** Biểu diễn dữ liệu trên một mặt phẳng Euclid. Thường được sử dụng cho các bản đồ vùng nhỏ hoặc khi các phép tính toán (như khoảng cách, diện tích) đã được chiếu lên một lưới phẳng cụ thể (ví dụ: hệ tọa độ VN-2000 của Việt Nam hoặc UTM).
- **Geography:** Biểu diễn dữ liệu trực tiếp trên mô hình mặt cong của Trái Đất. Các phép tính khoảng cách trên kiểu dữ liệu này sử dụng đường trắc địa, mang lại độ chính xác rất cao trên phạm vi lớn nhưng chi phí tính toán lớn hơn cấu trúc phẳng.

2.4.2.2. Các kiểu dữ liệu không gian cốt lõi

Trong dự án “Phát triển ứng dụng bản đồ số hỗ trợ sinh viên tìm phòng học trong khuôn viên Đại học Trà Vinh”, PostGIS cung cấp các thực thể hình học cơ bản bao gồm:

- **POINT:** Biểu diễn một cặp tọa độ (X, Y) hoặc (Long, Lat). Được dùng để đánh dấu các điểm mốc hoặc các nút giao thông trên sơ đồ đường đi.
- **LINESTRING:** Tập hợp chuỗi tuần tự của các điểm nối lại với nhau. Thể hiện các đoạn đường di chuyển nội khu, hành lang hoặc các tuyến đường bao quanh tòa nhà.
- **POLYGON:** Một chuỗi đường khép kín bao quanh một diện tích nhất định. Dùng để số hóa ranh giới các khối nhà, bãi đỗ xe, hồ nước hoặc các khu vực tiện ích.

2.4.2.3. Hệ quy chiếu tọa độ (SRID)

Một khái niệm nền tảng trong PostGIS là SRID. Mỗi thực thể không gian lưu trữ bắt buộc phải gắn liền với một mã hiệu SRID cụ thể để xác định cách định vị nó trên bề mặt Trái Đất. Trong các ứng dụng Web GIS hiện đại kết hợp với thư viện bản đồ như Leaflet hay Mapbox, hệ tọa độ toàn cầu SRID 4326 (WGS 84) sử dụng độ làm đơn vị

đo góc hoặc SRID 3857 (Web Mercator) được áp dụng phổ biến nhất để đồng bộ dữ liệu bản đồ nền trực tuyến.

2.4.2.4. Chỉ mục không gian và Cơ chế R-Tree

Đối với cơ sở dữ liệu thông thường, chỉ mục B-Tree hoạt động hiệu quả trên dữ liệu một chiều. Tuy nhiên, dữ liệu không gian là đa chiều, đòi hỏi PostGIS phải sử dụng cấu trúc chỉ mục chuyên dụng GiST dựa trên thuật toán R-Tree.

Cơ chế R-Tree hoạt động bằng cách chia nhỏ không gian và bọc các đối tượng hình học phức tạp bên trong các Hình hộp bao tối thiểu (MBR) theo cấu trúc phân cấp cây. Khi thực hiện các câu lệnh truy vấn tìm kiếm vị trí hoặc kiểm tra va chạm, PostGIS sẽ kiểm tra sự giao cắt giữa các hình hộp bao này trước. Nhờ loại bỏ nhanh chóng hàng loạt đối tượng không nằm trong vùng tìm kiếm mà không cần quét qua hình dạng hình học thô (thường rất phức tạp), tốc độ truy vấn không gian được tối ưu hóa tăng lên hàng nghìn lần.

2.4.2.5. Các hàm phân tích và xử lý không gian

PostGIS cung cấp hàng trăm hàm xử lý nội tại mạnh mẽ được tối ưu bằng C/C++, bao gồm ba nhóm chính:

- **Hàm quan hệ không gian:** Trả về kết quả Boolean để kiểm tra mối quan hệ giữa hai đối tượng, tuân theo mô hình 9-giao cắt mở rộng (DE-9IM). Ví dụ: ST_Contains(A, B): Kiểm tra hình A có chứa hình B không, ST_Intersects(A, B): Kiểm tra A và B có giao nhau không.
- **Hàm xử lý và đo đạc không gian:** Đo đạc các thông số vật lý như ST_Distance(A, B): Tính khoảng cách ngắn nhất giữa hai đối tượng, ST_Area(A): Tính diện tích vùng đa giác.
- **Hàm tạo hình học mới:** Trả về một đối tượng hình học mới từ dữ liệu đầu vào, chẳng hạn như ST_Buffer: Tạo vùng đệm xung quanh một điểm hoặc đường, hoặc ST_Centroid: Tìm tọa độ trọng tâm của một tòa nhà.

2.4.3. Tiện ích mở rộng pgRouting

pgRouting mở rộng khả năng của PostGIS bằng cách cung cấp các thuật toán định tuyến trên đồ thị trực tiếp bằng các truy vấn SQL.

- pgRouting phân tích cấu trúc cấu hình đường đi từ các bảng topo của PostGIS (với các trường cốt lõi như source, target, cost, reverse_cost).
- pgRouting tích hợp sẵn các hàm tối ưu hóa như pgr_astar, cho phép thực thi thuật toán A* ngay tại tầng Database, tận dụng tối đa năng lực xử lý phần cứng của máy chủ dữ liệu thay vì kéo toàn bộ dữ liệu thô về xử lý ở tầng ứng dụng Client hoặc Server-backend.

2.5. Mô hình MVC

Mô hình MVC (Model-View-Controller) là một mẫu kiến trúc phần mềm được giới thiệu lần đầu tiên bởi Trygve Reenskaug vào năm 1978 khi ông đang làm việc trên hệ thống Smalltalk-80 tại trung tâm nghiên cứu Xerox PARC. Mục tiêu cốt lõi của MVC là phân tách một ứng dụng thành ba thành phần độc lập tương tác với nhau, giúp hiện thực hóa nguyên lý Phân tách mối quan tâm SoC trong kỹ thuật phần mềm.

2.5.1. Các thành phần trong mô hình MVC

Kiến trúc MVC chia hệ thống logic của ứng dụng thành ba lớp chức năng biệt lập:

- **Model:** Đây là thành phần trung tâm của kiến trúc, chịu trách nhiệm quản lý toàn bộ trạng thái dữ liệu, các thực thể và quy tắc nghiệp vụ của ứng dụng. Model trực tiếp giao tiếp với Hệ quản trị cơ sở dữ liệu (ví dụ: PostgreSQL) để thực hiện các thao tác CRUD. Nó hoàn toàn độc lập với giao diện người dùng; Model không cần biết dữ liệu sẽ được hiển thị như thế nào.
- **View:** Là thành phần chịu trách nhiệm định dạng và hiển thị dữ liệu từ Model cho người dùng cuối dưới dạng giao diện trực quan. Trong các ứng dụng Web hiện đại, View nhận dữ liệu (thường dưới dạng JSON/Object) từ Controller để render ra các thành phần HTML/CSS hoặc thông qua các framework client-side

như Vue.js. View không chứa logic xử lý nghiệp vụ phức tạp mà chỉ tập trung vào trải nghiệm người dùng.

- **Controller:** Đóng vai cầu nối trung gian điều phối dòng làm việc giữa Model và View. Controller tiếp nhận các yêu cầu HTTP Requests gửi lên từ người dùng thông qua View hoặc các API Endpoint. Sau đó, nó phân tích yêu cầu, gọi các hàm logic tương ứng ở Model để xử lý dữ liệu, rồi chọn lựa View thích hợp (hoặc trả về chuỗi JSON dữ liệu) để phản hồi lại cho Client.

2.5.2. Luồng xử lý trong MVC

Trong môi trường phát triển web, luồng xử lý của mô hình MVC diễn ra theo chu trình sau:

- **Gửi yêu cầu:** Người dùng tương tác với giao diện (View), thực hiện một hành động. Trình duyệt sẽ gửi một HTTP Request kèm theo dữ liệu đầu vào đến Server.
- **Tiếp nhận và Điều phối:** Bộ định tuyến (Router) của Server tiếp nhận yêu cầu và chuyển hướng đến đúng Controller chịu trách nhiệm xử lý.
- **Xử lý nghiệp vụ:** Controller tiếp nhận dữ liệu đầu vào, tiến hành kiểm tra tính hợp lệ (Validation). Sau đó, nó gọi các phương thức tương ứng của Model để thực hiện tính toán hoặc truy vấn xuống Database.
- **Cập nhật dữ liệu:** Model thực hiện xử lý dữ liệu, cập nhật trạng thái mới vào Database và trả kết quả thô về cho Controller.
- **Chuẩn bị phản hồi:** Controller tiếp nhận kết quả từ Model. Đối với kiến trúc web truyền thống, nó sẽ truyền dữ liệu này vào View để render ra giao diện HTML. Đối với kiến trúc Web API hiện đại, Controller sẽ đóng gói dữ liệu thành định dạng JSON và gửi thẳng về phía Client để Vue tự cập nhật giao diện.
- **Hiển thị:** Người dùng nhận được phản hồi trực quan trên màn hình trình duyệt.

2.5.3. Ưu và Nhược điểm của mô hình kiến trúc MVC

Ưu điểm:

- **Tính độc lập và Tái sử dụng cao:** Vì các thành phần được phân tách rõ rệt, lập trình viên có thể thay đổi hoàn toàn giao diện (View) hoặc chuyển đổi hệ quản trị cơ sở dữ liệu (Model) mà không cần phải viết lại logic điều khiển của hệ thống.
- **Hỗ trợ phát triển song song:** Trong một nhóm dự án, các lập trình viên Frontend có thể tập trung thiết kế giao diện ứng dụng trên Vue.js (View), trong khi các lập trình viên Backend có thể đồng thời xây dựng logic xử lý API trên Node.js (Model và Controller) mà không bị phụ thuộc hay gây ảnh hưởng lẫn nhau.
- **Dễ dàng bảo trì và Mở rộng:** Khi hệ thống phát sinh lỗi hoặc cần nâng cấp thêm tính năng mới, cấu trúc rõ ràng của MVC giúp việc định vị mã nguồn lỗi và tích hợp module mới trở nên nhanh chóng, giảm thiểu tối đa rủi ro xung đột mã.
- **Thân thiện với kiểm thử:** Do tách biệt phần xử lý logic ra khỏi phần hiển thị giao diện, việc viết các kịch bản kiểm thử tự động cho các hàm tính toán của Model hay Controller trở nên dễ dàng và đạt độ chính xác cao hơn.

Nhược điểm:

- **Tăng độ phức tạp của mã nguồn hệ thống:** Đối với các ứng dụng quy mô nhỏ hoặc các trang web tĩnh đơn giản, việc áp dụng cấu trúc MVC bắt buộc phải chia nhỏ tệp tin, phân cấp thư mục phức tạp, dẫn đến lãng phí thời gian thiết lập ban đầu và tăng lượng mã nguồn không cần thiết.
- **Độ trễ trong việc cập nhật dữ liệu liên tục:** Việc dữ liệu phải đi qua chu trình tuần hoàn nhiều bước có thể tạo ra một độ trễ nhất định. Với các tác vụ yêu cầu phản hồi đồ họa thời gian thực siêu tốc độ, cấu trúc phân lớp chặt chẽ của MVC đôi khi làm giảm hiệu năng hệ thống.
- **Yêu cầu kỹ năng chuyên môn cao:** Đòi hỏi các thành viên trong đội ngũ phát triển phải có tư duy thiết kế hệ thống tốt, hiểu rõ ranh giới trách nhiệm của từng lớp (Model, View, Controller) để tránh việc viết code sai vị trí.

2.6. Docker

2.6.1. Cơ bản về Docker

Docker là một nền tảng mã nguồn mở cho phép tự động hóa quy trình triển khai, quản lý và vận hành ứng dụng dưới dạng các gói phần mềm biệt lập được gọi là Container. Thay vì ảo hóa toàn bộ phần cứng như công nghệ máy ảo, Docker sử dụng cơ chế ảo hóa ở cấp độ hệ điều hành.

Các thành phần cốt lõi cấu thành nên hệ sinh thái Docker bao gồm:

- **Docker Engine:** Thành phần trung tâm, đóng vai trò là một ứng dụng client-server chịu trách nhiệm khởi tạo, quản lý và vận hành các container.
- **Docker Image:** Một khuôn mẫu đóng băng, chứa toàn bộ mã nguồn, thư viện hệ thống, biến môi trường và các tệp cấu hình cần thiết để ứng dụng có thể chạy được.
- **Docker Container:** Một thực thể sống được khởi tạo từ một Docker Image. Mỗi container hoạt động hoàn toàn biệt lập với môi trường máy host và các container khác, sở hữu không gian tiến trình và tài nguyên mạng riêng.
- **Docker Compose:** Công cụ hỗ trợ định nghĩa và vận hành các ứng dụng phức tạp chạy đa container thông qua một tệp cấu hình tập trung duy nhất (docker-compose.yml).

2.6.2. Ưu, nhược điểm

Ưu điểm:

- **Nhẹ và Tiết kiệm tài nguyên:** Khác với máy ảo đòi hỏi một hệ điều hành khách riêng biệt và tiêu tốn lượng lớn RAM/CPU nền, các Docker Container chia sẻ chung nhân hệ điều hành của máy host. Do đó, kích thước tệp và lượng tài nguyên phần cứng tiêu thụ được giảm thiểu tối đa.
- **Tính nhất quán của môi trường:** Docker giải quyết triệt để bài toán xung đột phiên bản thư viện hoặc sai lệch môi trường giữa máy lập trình của nhà phát triển và máy chủ vận hành thực tế. Một khi Image đã chạy ổn định, nó chắc chắn sẽ hoạt động đồng nhất trên mọi hạ tầng hỗ trợ Docker.

-
- **Quản lý cấu hình tinh gọn:** Thông qua Dockerfile và Docker Compose, toàn bộ cơ sở hạ tầng của phần mềm được mã hóa thành văn bản, giúp việc cấu hình, bàn giao dự án hoặc mở rộng quy mô hệ thống trở nên cực kỳ dễ dàng và minh bạch.

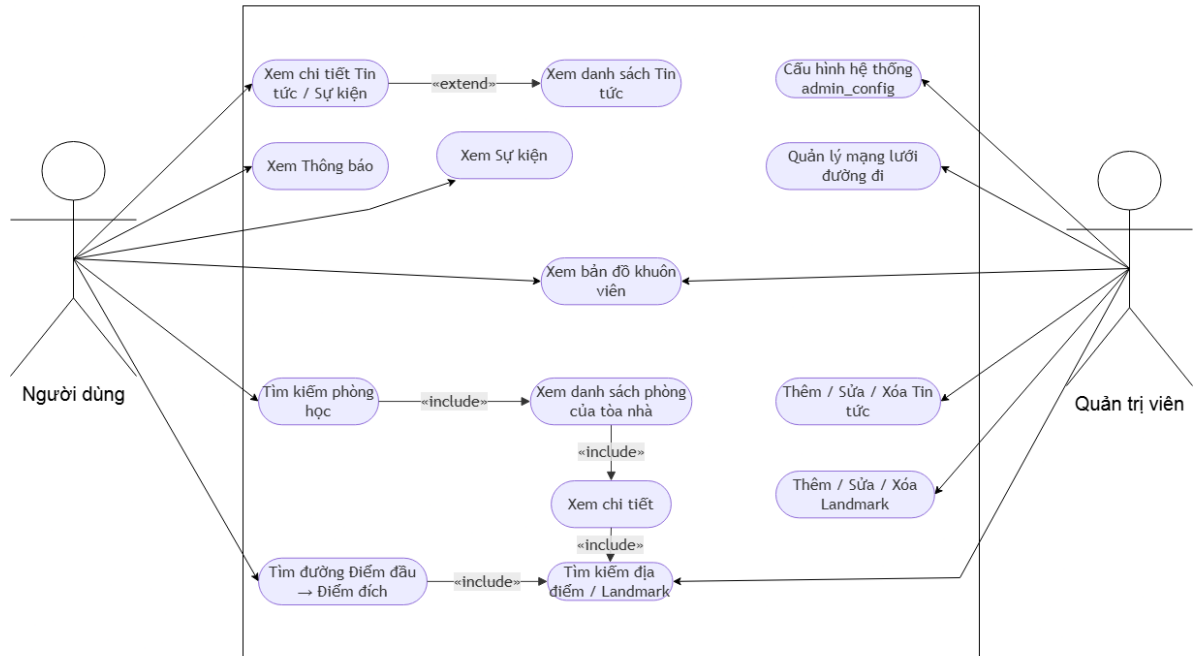
Nhược điểm:

- **Giới hạn về tính bảo mật:** Vì các container chia sẻ chung nhân hệ điều hành của máy host, mức độ cô lập an toàn thông tin của Docker thấp hơn so với máy ảo. Nếu có một lỗ hổng bảo mật nghiêm trọng khai thác được nhân hệ điều hành, tất cả các container trên máy host đó đều có nguy cơ bị ảnh hưởng.
- **Bất lợi về hiệu năng đối với tác vụ nặng:** Mặc dù chạy nhanh hơn máy ảo, nhưng dữ liệu truyền qua các lớp mạng ảo và lớp lưu trữ dữ liệu của Docker vẫn có một độ trễ nhất định so với việc ứng dụng chạy trực tiếp trên hệ điều hành máy host.
- **Khó khăn trong lưu trữ dữ liệu bền vững:** Theo thiết kế mặc định, container mang tính chất tạm thời. Khi container bị xóa, toàn bộ dữ liệu tạo ra bên trong nó cũng biến mất. Do đó, việc cấu hình lưu trữ dữ liệu cho các hệ quản trị cơ sở dữ liệu lớn (như PostgreSQL) đòi hỏi kỹ thuật quản lý volume phức tạp để đảm bảo dữ liệu không bị thất thoát.

CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Thiết kế cơ sở dữ liệu

3.1.1. Lược đồ use case tổng quan của hệ thống

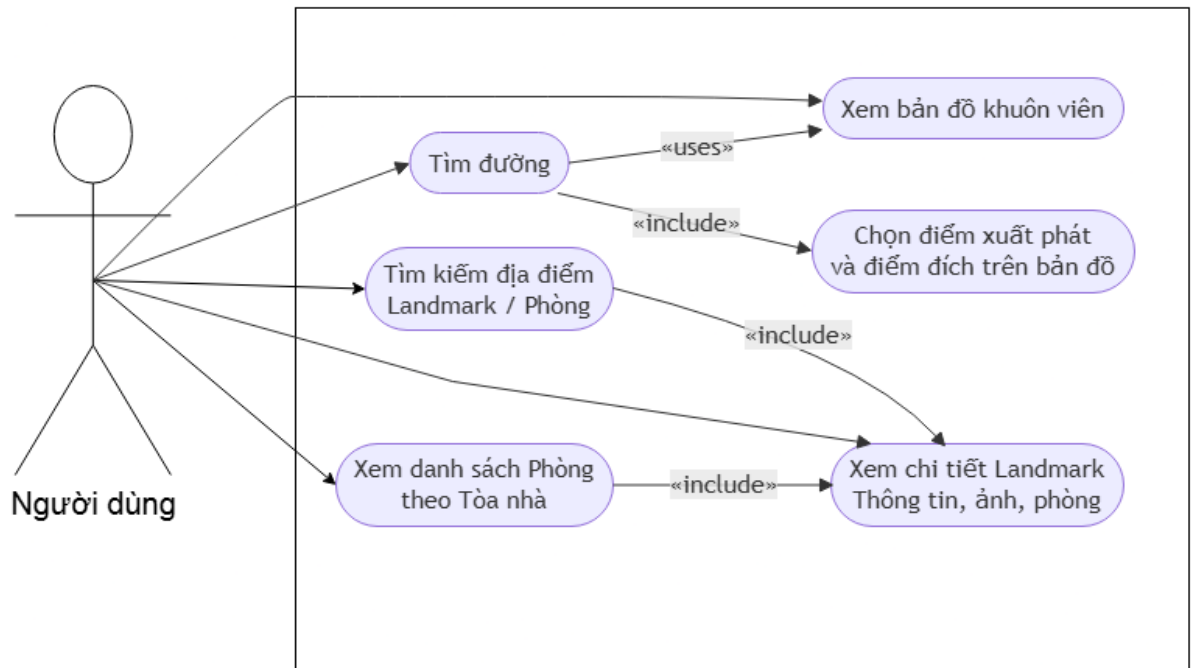


Hình 3.1 Mô hình use case tổng quan của hệ thống

Mô tả: Người dùng có quyền xem bản đồ khuôn viên trường, tìm kiếm tòa nhà và phòng học, tìm đường đến phòng học. Quản trị viên có quyền quản lý bài đăng, quản lý các tuyến đường, quản lý thông tin tòa nhà.

3.1.2. Mô tả các use case

3.1.2.1. Use case xem bản đồ



Hình 3.2 Mô hình use case người dùng sử dụng bản đồ

Xem bản đồ khuôn viên: Cho phép người dùng tiếp cận giao diện trực quan của toàn bộ khuôn viên trường/tòa nhà. Đây là tính năng nền tảng phục vụ cho các tương tác không gian tiếp theo.

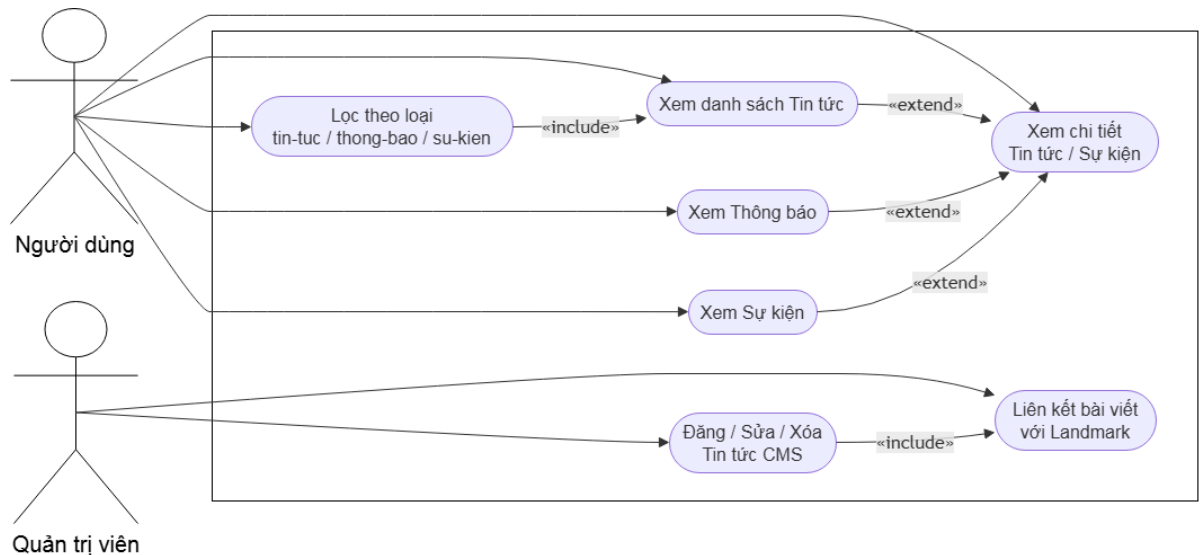
Tìm đường: Người dùng có thể tìm kiếm tuyến đường tối ưu giữa hai điểm.

- Mối quan hệ «uses»: Chức năng này bắt buộc phải sử dụng, hiển thị giao diện dữ liệu trên **Xem bản đồ khuôn viên**.
- Mối quan hệ «include»: Bắt buộc người dùng phải tiến hành **Chọn điểm xuất phát và điểm đích trên bản đồ** để hệ thống có cơ sở tính toán định vị.

Tìm kiếm địa điểm Landmark / Phòng: Người dùng nhập từ khóa để truy vấn nhanh vị trí. Chức năng này tích hợp mối quan hệ «include» đến chức năng **Xem chi tiết Landmark** để hiển thị trực quan thông tin chi tiết ngay sau khi tìm thấy kết quả.

Xem danh sách Phòng theo Tòa nhà: Hỗ trợ duyệt dữ liệu dạng phân cấp. Chức năng này cũng tự động «include» chức năng **Xem chi tiết Landmark** khi người dùng chọn một phòng/địa điểm cụ thể từ danh sách.

3.1.2.2. Use case xem bài đăng



Hình 3.3 Mô hình use case xem bài đăng

Phía tác nhân Người dùng:

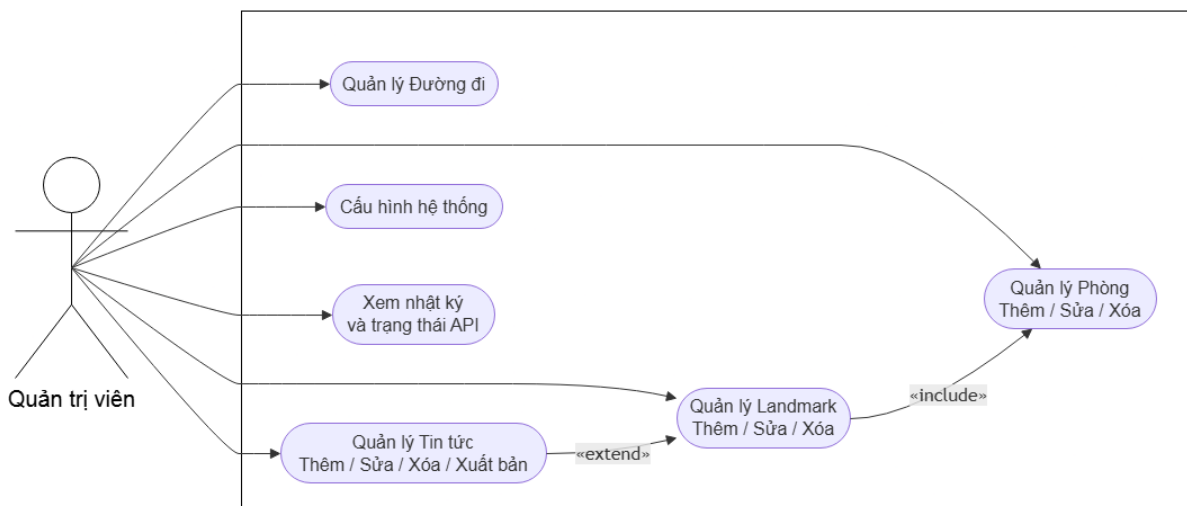
- **Lọc theo loại (tin tức / thông báo / sự kiện):** Người dùng chủ động phân loại luồng thông tin theo nhu cầu. Chức năng này «include» việc Xem danh sách Tin tức tương ứng sau khi bộ lọc được áp dụng.
- **Xem chi tiết Tin tức / Sự kiện:** Đây là đích đến cuối cùng của luồng đọc tin. Chức năng này là phần mở rộng của ba hành động độc lập: Xem danh sách Tin tức, Xem Thông báo, và Xem Sự kiện. Khi người dùng có nhu cầu tìm hiểu sâu, một yêu cầu gọi trang chi tiết sẽ được kích hoạt.

Phía tác nhân Quản trị viên:

- **Đăng / Sửa / Xóa Tin tức CMS:** Tính năng thuộc trang quản trị, cho phép thực hiện các thao tác CRUD cơ bản trên thực thể Tin tức.

- **Liên kết bài viết với Landmark:** Là chức năng bắt buộc trong luồng quản lý tin tức hoặc có thể thao tác độc lập từ phía quản trị viên, nhằm mục đích định vị không gian cho tin tức.

3.1.2.3. Use case quản trị viên



Hình 3.4 Mô hình use case quản trị viên

Quản lý Đường đi: Cho phép thiết lập, cập nhật đồ thị giao thông nội khu nhằm phục vụ thuật toán tìm đường của người dùng.

Cấu hình hệ thống: Thiết lập các tham số môi trường, biến cấu hình hệ thống, quản lý quyền hạn hoặc các thông số vận hành tổng thể.

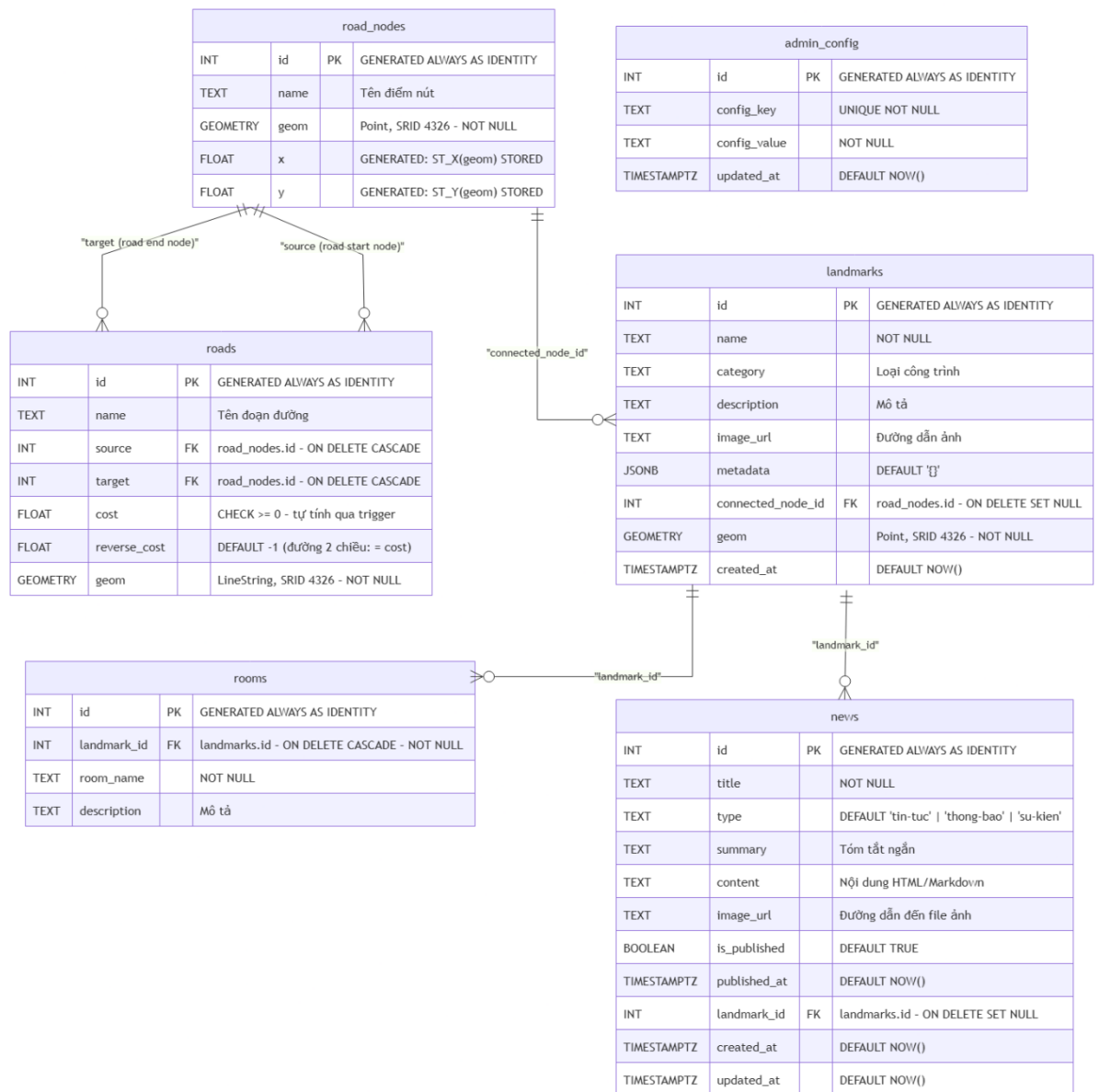
Xem nhật ký và trạng thái API: Chức năng giám sát dành cho kỹ thuật viên nhằm theo dõi lịch sử hoạt động, phát hiện lỗi phát sinh và kiểm tra hiệu năng hệ thống của các cổng API kết nối.

Quản lý Phòng (Thêm / Sửa / Xóa): Quản trị thực thể Phòng học/Phòng chức năng trong hệ thống. Chức năng này có thể tương tác trực tiếp từ tác nhân Quản trị viên.

Quản lý Landmark (Thêm / Sửa / Xóa): Quản trị thực thể không gian lớn hơn. Chức năng này bắt buộc «include» chức năng Quản lý Phòng, đảm bảo tính toàn vẹn dữ liệu (khi tác động đến Landmark sẽ ảnh hưởng hoặc yêu cầu cấu hình các phòng bên trong thuộc Landmark đó).

Quản lý Tin tức (Thêm / Sửa / Xóa / Xuất bản): Tính năng tổng quan về quản trị nội dung. Hành động này có mối quan hệ «extend» hướng tới Quản lý Landmark (trong kịch bản quản trị viên muốn gán trực tiếp hoặc điều chỉnh thông tin địa điểm liên quan ngay trong quá trình thêm / sửa tin tức).

3.1.3. Mô hình cơ sở dữ liệu ERD



Hình 3.5 Mô hình cơ sở dữ liệu ERD

Để phục vụ cho các thuật toán tìm đường, mạng lưới đường đi được mô hình hóa thành một đồ thị có hướng. Do đó, bảng roads cần hai liên kết đồng thời đến bảng road_nodes:

- **Trường source (Khóa ngoại):** Lưu trữ ID của nút giao nơi đoạn đường bắt đầu.
- **Trường target (Khóa ngoại):** Lưu trữ ID của nút giao nơi đoạn đường kết thúc

Việc tách biệt hai liên kết này là bắt buộc để hệ thống xác định được hướng di chuyển, từ đó kết hợp với các trường chi phí (cost và reverse_cost) nhằm tính toán chính xác lộ trình di chuyển (bao gồm cả các tuyến đường một chiều hoặc hai chiều).

3.1.4. Chi tiết các thực thể

3.1.4.1. Thực thể road_nodes

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh của nút giao	INT	PK
2	name	Tên của nút	TEXT	
3	geom	Tọa độ không gian của điểm	GEOMETRY (Point, 4326)	NOT NULL, Hệ tọa độ WGS 84 (SRID 4326)
4	x	Kinh độ	FLOAT	GENERATED ALWAYS AS (ST_X(geom)) STORED
5	y	Vĩ độ	FLOAT	GENERATED ALWAYS AS (ST_Y(geom)) STORED

Bảng 3.1 Bảng thực thể road_nodes

Mô tả: Trong mô hình hóa mạng lưới giao thông phục vụ thuật toán tìm đường, thực thể road_nodes đóng vai trò là tập hợp các đỉnh. Mỗi bản ghi trong bảng đại diện cho một điểm nút không gian cố định trong khuôn viên trường Đại học Trà Vinh, bao

gồm các vị trí như nút giao giữa các tuyến đường, khúc cua, điểm rẽ hoặc các điểm chuyển tiếp.

3.1.4.2. Thực thể roads

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh của nút giao	INT	PK
2	name	Tên của nút	TEXT	
3	source	Điểm bắt đầu của đoạn đường	INT	FK
4	target	Điểm kết thúc của đoạn đường	INT	FK
5	cost	Chi phí di chuyển chiều thuận	FLOAT	
6	reverse_cost	Chi phí di chuyển chiều nghịch	FLOAT	
7	geom	Tọa độ không gian của đường	GEOMETRY(Point, 4326)	NOT NULL

Bảng 3.2 Bảng thực thể roads

Mô tả: Thực thể roads đại diện cho tập hợp các cạnh trong cấu trúc đồ thị mạng lưới đường đi nội khu. Mỗi bản ghi thiết lập một liên kết không gian và logic tuyến tính giữa hai đỉnh cụ thể (được xác định qua cặp khóa ngoại source và target tham chiếu đến thực thể road_nodes).

3.1.4.3. Thực thể landmarks

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh duy nhất của địa điểm	INT	PK
2	name	Tên địa điểm	TEXT	NOT NULL
3	category	Danh mục địa điểm	TEXT	
4	description	Mô tả chi tiết về địa điểm	TEXT	
5	image_url	Đường dẫn ảnh đại diện của địa điểm	TEXT	
6	metadata	Dữ liệu mở rộng dạng cấu trúc JSON	JSONB	
7	connected_node_id	Nút giao gần nhất được kết nối để tìm đường	INT	FK
8	geom	Tọa độ vị trí của địa điểm	GEOMETRY(Point, 4326)	NOT NULL
9	created_at	Thời gian tạo địa điểm	TIMESTAMPTZ	DEFAULT NOW()

Bảng 3.3 Bảng thực thể landmarks

Mô tả: Thực thể landmarks đóng vai trò lưu trữ các đối tượng điểm thu hút và các thực thể kiến trúc nổi bật trong khuôn viên trường (như các tòa nhà, giảng đường, trung tâm hành chính, thư viện, căn tin). Thực thể này cầu nối giữa dữ liệu hệ thống thông tin địa lý và dữ liệu định danh đối tượng thuộc thể giới thực.

3.1.4.4. Thực thể rooms

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh của phòng	INT	PK
2	landmark_id	Thuộc về tòa nhà nào	INT	FK
3	room_name	Tên hoặc số phòng	TEXT	NOT NULL
4	description	Mô tả về phòng	TEXT	

Bảng 3.4 Bảng thực thể rooms

Mô tả: Thực thể rooms được thiết kế nhằm cụ thể hóa cấu trúc phân cấp không gian bên trong các tòa nhà (Landmarks). Trong một mốc bản đồ hay tòa nhà tổng thể, sẽ tồn tại các phòng học, phòng thực hành, phòng làm việc của giảng viên hoặc phòng ban chức năng.

3.1.4.5. Thực thể news

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh duy nhất của bài viết	INT	PK
2	title	Tiêu đề bài viết	TEXT	NOT NULL
3	type	Phân loại bài viết	TEXT	NOT NULL
4	summary	Tóm tắt ngắn hiển thị ngoài danh sách	TEXT	
5	content	Nội dung chi tiết bài viết (HTML/Markdown)	TEXT	
6	image_url	Ảnh đại diện của bài đăng	TEXT	
7	is_published	Trạng thái xuất bản (Ẩn/Hiện bản nháp)	BOOLEAN	DEFAULT TRUE
8	published_at	Thời gian xuất bản bài viết	TIMESTAMPTZ	DEFAULT NOW()

9	landmark_id	Gắn bài viết/sự kiện này vào một địa điểm	INT	FK
10	created_at	Thời gian tạo bài viết	TIMESTAM PTZ	DEFAULT NOW()
11	updated_at	Thời gian cập nhật bài viết mới nhất	TIMESTAM PTZ	DEFAULT NOW()

Bảng 3.5 Bảng thực thể news

Mô tả: Thực thể news đóng vai trò quản lý và lưu trữ hệ thống thông tin truyền thông, các sự kiện học thuật, thông báo nội bộ và tin tức định kỳ của nhà trường. Khác biệt với các hệ thống CMS truyền thông thông thường, thực thể này được tích hợp đặc tính "nhận thức không gian" nhờ mối liên kết với các thực thể địa lý cụ thể trên bản đồ số.

3.1.4.6. Thực thể admin_config

STT	Thuộc tính	Diễn giải	Kiểu dữ liệu	Ràng buộc
1	id	Mã định danh duy nhất của cấu hình	INT	PK
2	config_key	Tên từ khóa cấu hình (Ví dụ: site_title)	TEXT	NOT NULL, UNIQUE
3	config_value	Giá trị tương ứng của từ khóa	TEXT	NOT NULL
4	updated_at	Thời gian cập nhật cấu hình	TIMESTAM PTZ	DEFAULT NOW()

Bảng 3.6 Bảng thực thể admin_config

Mô tả: Thực thể admin_config đóng vai trò là kho lưu trữ tập trung các tham số thiết lập môi trường, biến cấu hình toàn cục quản trị vận hành ứng dụng.

3.2. Xây dựng Backend REST API

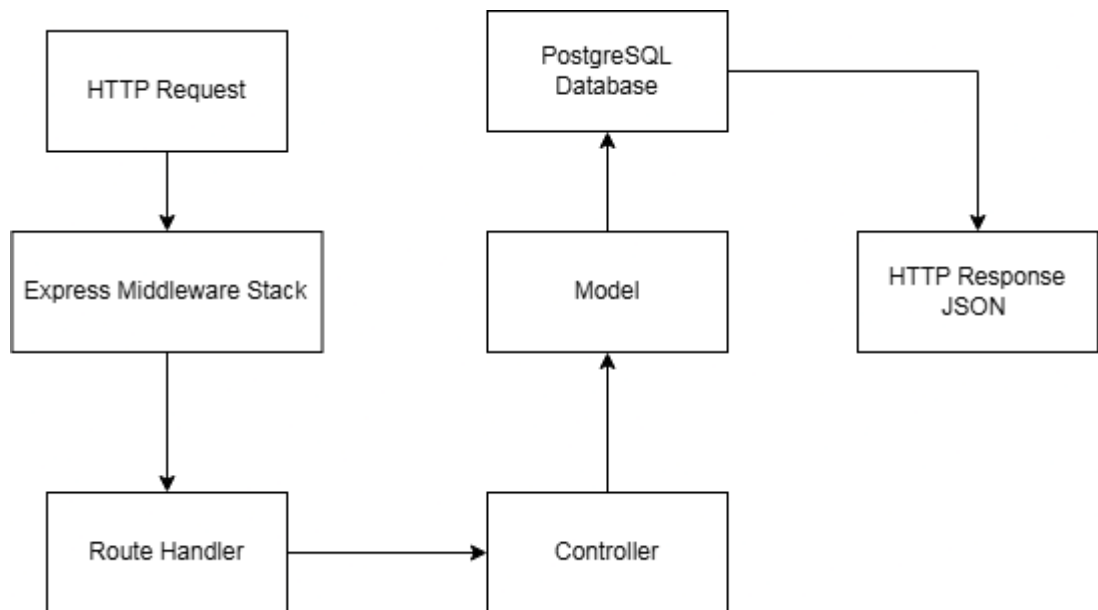
3.2.1. Xây dựng Backend theo mô hình MVC

Tầng ứng dụng (Backend) của hệ thống được tổ chức dựa trên kiến trúc MVC thu gọn. Trong cấu trúc này, ba thành phần cốt lõi của mô hình Model-View-Controller được hiện thực hóa cụ thể như sau:

-
- **Tầng Model:** Đảm nhận nhiệm vụ truy vấn và thao tác dữ liệu trực tiếp với cơ sở dữ liệu, được tổ chức trong thư mục models.
 - **Tầng Controller:** Đóng vai trò xử lý logic nghiệp vụ (business logic) và định dạng các phản hồi kết quả, được tổ chức trong thư mục controllers.
 - **Tầng View:** Do hệ thống Backend được thiết kế theo mô hình API Server thuần túy (không đảm nhận chức năng kết xuất giao diện HTML), tầng View truyền thống được thay thế hoàn toàn bằng các cấu trúc phản hồi định dạng JSON thuần túy.

Bên cạnh ba thành phần chính, cơ chế điều hướng yêu cầu (routing) được tách biệt thành một phân hệ độc lập (nằm trong thư mục routes). Phân hệ này đóng vai trò trung gian, ánh xạ các điểm cuối URL đến các phương thức xử lý tương ứng trong tầng Controller.

Luồng xử lý một yêu cầu HTTP Request hoàn chỉnh trong hệ thống được thực hiện theo chuỗi tuần tự sau:



Hình 3.6 Sơ đồ luồng xử lý yêu cầu HTTP tại Backend

Nhằm đảm bảo tính nhất quán của toàn bộ hệ thống API, tất cả các phản hồi từ Server đều tuân thủ cấu trúc dữ liệu JSON chuẩn hóa bao gồm các trường thông tin:

- **success (kiểu dữ liệu logic - boolean):** Chỉ thị trạng thái thành công hoặc thất bại của thao tác.
- **data (kiểu dữ liệu mảng hoặc đối tượng):** Chứa gói tin kết quả.
- **message (kiểu dữ liệu chuỗi - string):** Cung cấp mô tả chi tiết về trạng thái thành công hoặc thông báo lỗi cụ thể.

Thiết kế đồng bộ này hỗ trợ tầng ứng dụng phía Client xây dựng một cơ chế xử lý phản hồi hợp nhất, giảm thiểu mã nguồn trùng lặp.

Một đặc điểm kỹ thuật quan trọng trong kiến trúc nguồn của Backend là việc áp dụng tiêu chuẩn **ES Module** (khai báo type: "module" trong tệp cấu hình package.json), thay thế cho chuẩn CommonJS truyền thống (require và module.exports). Giải pháp này cho phép hệ thống khai thác cú pháp import/export hiện đại, tối ưu hóa kích thước mã nguồn, đồng thời đồng nhất kiến trúc module giữa hai tầng Frontend và Backend trong mô hình quản lý mã nguồn tập trung.

3.2.2. Xây dựng REST API

3.2.2.1. API Landmark

Phương thức	Điểm cuối	Tham số truy vấn	Mô tả chức năng
GET	/api/landmarks	—	Truy xuất toàn bộ danh sách địa điểm trong hệ thống.
GET	/api/landmarks	?category=xxx	Lọc danh sách địa điểm theo danh mục phân loại.
GET	/api/landmarks	?search=xxx	Tìm kiếm địa điểm theo tên hoặc mô tả.
GET	/api/landmarks	?near=lng,lat&limit=5	Tìm kiếm N địa điểm có vị trí không gian gần nhất.
GET	/api/landmarks/categories	—	Truy xuất danh sách các danh mục phân loại duy nhất.
GET	/api/landmarks/:id	—	Truy xuất thông tin chi tiết của một địa

			điểm theo định danh.
POST	/api/landmarks	—	Khởi tạo một bản ghi địa điểm mới vào hệ thống.
PUT	/api/landmarks/:id	—	Cập nhật thông tin của địa điểm hiện tại theo định danh.
DELETE	/api/landmarks/:id	—	Loại bỏ địa điểm ra khỏi cơ sở dữ liệu hệ thống.

Bảng 3.7 Danh mục các API Landmark

3.2.2.2. API Rooms

Phương thức	Điểm cuối	Tham số truy vấn	Mô tả chức năng
GET	/api/rooms	—	Truy xuất toàn bộ danh sách phòng học hiện có.
GET	/api/rooms	?landmark_id=N	Lọc danh sách phòng học theo mã định danh tòa nhà.
GET	/api/rooms	?search=xxx	Tìm kiếm phòng học theo chuỗi ký tự tên phòng.
GET	/api/rooms/:id	—	Truy xuất thông tin chi tiết của một phòng học cụ thể.
POST	/api/rooms	—	Khởi tạo phòng học mới (yêu cầu tham số landmark id).
PUT	/api/rooms/:id	—	Cập nhật dữ liệu thuộc tính của phòng học hiện tại.
DELETE	/api/rooms/:id	—	Xóa bản ghi phòng học khỏi hệ thống.

Bảng 3.8 Danh mục các API Rooms

3.2.2.3. API Paths

Phương thức	Điểm cuối	Tham số truy vấn	Mô tả chức năng
GET	/api/paths/find	?from=N&to=M	Tính toán đường đi tối ưu theo thuật toán A* giữa hai nút giao thông.
GET	/api/paths	?geojson=true	Xuất dữ liệu mạng lưới đường dưới dạng định dạng chuẩn GeoJSON.
GET	/api/paths/:id	—	Truy xuất thông tin thuộc tính của một đoạn đường cụ thể.

POST	/api/paths	—	Khởi tạo một đoạn đường mới vào mô hình mạng lưới.
PUT	/api/paths/:id	—	Cập nhật thuộc tính hình học hoặc thông tin đoạn đường.
DELETE	/api/paths/:id	—	Loại bỏ đoạn đường khỏi mô hình đồ thị.
GET	/api/paths/nodes	?near=lng,lat	Xác định nút giao thông có khoảng cách gần nhất với tọa độ chỉ định.
GET	/api/paths/nodes/:id	—	Truy xuất chi tiết thông tin một nút giao thông cụ thể.
POST	/api/paths/nodes	—	Khởi tạo một điểm nút giao thông mới.
PUT	/api/paths/nodes/:id	—	Cập nhật tọa độ không gian hoặc thuộc tính của nút.
DELETE	/api/paths/nodes/:id	—	Xóa điểm nút giao thông khỏi mô hình đồ thị mạng lưới.

Bảng 3.9 Danh mục các API Paths

3.2.2.4. API News

Phương thức	Điểm cuối	Tham số truy vấn	Mô tả chức năng
GET	/api/news	?page&limit	Truy xuất danh sách bài viết tin tức kèm cơ chế phân trang dữ liệu.
GET	/api/news	?landmark_id=N	Lọc danh sách bài viết tin tức liên quan đến một địa điểm cụ thể.
GET	/api/news	?search=xxx	Tìm kiếm bài viết dựa theo từ khóa trong tiêu đề.
GET	/api/news/:id	—	Truy xuất nội dung thông tin chi tiết của bài viết tin tức.
POST	/api/news	—	Khởi tạo một bài viết tin tức mới (hỗ trợ đính kèm tệp tin hình ảnh).
PUT	/api/news/:id	—	Cập nhật nội dung bài viết hiện có (hỗ trợ thay đổi tệp tin hình ảnh).
DELETE	/api/news/:id	—	Loại bỏ bản ghi bài viết tin tức khỏi cơ sở dữ liệu hệ thống.

Bảng 3.10 Danh mục các API News

3.3. Phát triển giao diện và thuật toán

3.3.1. Kiến trúc ứng dụng SPA với Vue 3 Composition API

Giao diện người dùng của hệ thống được phát triển theo mô hình Ứng dụng Đơn Trang (SPA). Trong mô hình này, toàn bộ mã nguồn giao diện được tải một lần duy nhất tại thời điểm khởi tạo; các phân hệ trang con sau đó sẽ được kết xuất động tại Client mà không yêu cầu tải lại toàn bộ trình duyệt. Giải pháp kiến trúc này tối ưu hóa hiệu năng đối với các ứng dụng bản đồ tương tác, đảm bảo quá trình chuyển đổi giữa các phân hệ diễn ra mượt mà, đồng thời duy trì liên tục trạng thái vận hành của bản đồ như tọa độ góc nhìn và các lớp dữ liệu không gian đang hiển thị.

Hệ thống khai thác nền tảng Vue 3 kết hợp phương thức lập trình Composition API (thông qua cấu trúc cú pháp <script setup>) thay thế cho mô hình Options API truyền thống. Phương thức Composition API cho phép tổ chức mã nguồn theo khối chức năng thay vì phân tách theo thuộc tính cú pháp, từ đó tạo điều kiện thuận lợi cho việc tái sử dụng mã nguồn thông qua các Composable Function. Các hàm chức năng này bản chất là các hàm JavaScript thuần túy, có nhiệm vụ tiếp nhận và phản hồi các trạng thái phản xạ, cho phép tích hợp linh hoạt vào bất kỳ thành phần giao diện nào trong hệ thống.

3.3.2. Cơ chế tích hợp nền bản đồ qua Leaflet

Thực thể bản đồ tương tác được thiết lập thông qua phương thức khởi tạo `initMap()`, được kích hoạt trong vòng đời hook `onMounted` ngay sau khi hệ thống hoàn tất quy trình tải dữ liệu không gian từ API Server. Tiến trình cấu hình được thực hiện theo các bước tuần tự:

```
function initMap() {  
  if (!mapContainer.value || map) return;  
  map = L.map(mapContainer.value, { zoomControl: false }).setView([9.92345, 106.34785], 17);  
  
  // Tạo sẵn cả 2 tile layers  
  streetTile = L.tileLayer(LAYERS.street.url, LAYERS.street.options);  
  satelliteTile = L.tileLayer(LAYERS.satellite.url, LAYERS.satellite.options);  
  
  // Thêm layer mặc định (street)  
  streetTile.addTo(map);  
  currentLayer.value = 'street';  
}
```

Hình 3.7 Phương thức thiết lập cấu hình bản đồ tương tác Leaflet

Hệ thống cung cấp cơ chế chuyển đổi linh hoạt giữa hai phân lớp bản đồ nền tùy theo nhu cầu khai thác thông tin:

- OpenStreetMap (tile.openstreetmap.org): Bản đồ dữ liệu đường giao thông mã nguồn mở, hỗ trợ hiển thị chi tiết hệ thống tên đường và các yếu tố địa lý tự nhiên.
- Esri World Imagery (server.arcgisonline.com): Bản đồ không ảnh vệ tinh độ phân giải cao, hỗ trợ người dùng nhận diện trực quan cấu trúc hình học thực tế của các công trình kiến trúc.

Thư viện Leaflet được định cấu hình vận hành mặc định trên hệ tọa độ địa lý toàn cầu WGS 84 (Mã định danh EPSG:4326), tiếp nhận mảng tham số tọa độ theo thứ tự vĩ độ trước - kinh độ sau [lat, lng]. Đây là một đặc điểm kỹ thuật quan trọng cần lưu ý, do hệ quản trị cơ sở dữ liệu PostgreSQL/PostGIS lưu trữ dữ liệu không gian theo quy chuẩn cấu trúc định dạng GeoJSON với thứ tự kinh độ trước - vĩ độ sau [lng, lat]. Do đó, hệ thống Client bắt buộc phải thực hiện bước hoán đổi thứ tự các phần tử tọa độ trước khi chuyển tiếp dữ liệu vào thực thể bản đồ Leaflet.

Biến tham chiếu bản đồ được khai báo dưới dạng biến ngữ cảnh let thông thường, chủ động không áp dụng thuộc tính phản xạ. Giải pháp thiết kế này nhằm mục đích ngăn chặn cơ chế kiểm soát phản xạ dữ liệu của Vue tác động lên thực thể Leaflet, bởi Leaflet sở hữu cơ chế tự quản lý trạng thái nội bộ. Việc bọc một đối tượng Proxy phản xạ của Vue lên trên thực thể Leaflet có khả năng gây ra các xung đột tài nguyên và làm suy giảm hiệu năng kết xuất đồ họa.

3.3.3. Thuật toán tìm đường

Tính năng tính toán và hiển thị đường đi tối ưu của hệ thống được thiết kế dựa trên chiến lược định tuyến đa tầng. Chiến lược này đảm bảo khả năng xử lý đồng thời hai kịch bản vị trí thực tế của người dùng: kịch bản người dùng đang di chuyển bên trong khuôn viên trường và kịch bản người dùng đang tiếp cận từ bên ngoài khuôn viên.

3.3.3.1. Module tìm đường A*

Phân hệ dịch vụ AStarService đóng vai trò là tầng dịch vụ trung gian kết nối các thành phần giao diện Frontend với các đầu cuối REST API tìm đường của Backend. Module cung cấp hệ thống các phương thức xử lý cốt lõi:

Phương thức isInsideCampus(lat, lng): Sử dụng mô hình hình học hộp bao quanh giới cố định để xác định trạng thái không gian của tọa độ hiện hành.

```
// Bounding box khuôn viên ĐH Trà Vinh (từ dữ liệu road.sql)
const CAMPUS_BOUNDS = {
  lngMin: 106.345,
  lngMax: 106.351,
  latMin: 9.918,
  latMax: 9.925,
};
```

Hình 3.8 Định nghĩa tọa độ giới hạn ranh giới khuôn viên ĐH Trà Vinh

Phương thức getCurrentPosition(): Triển khai chiến lược tiếp nhận dữ liệu vị trí theo cơ chế hai tầng dự phòng nhằm tối ưu hóa thời gian phản hồi:

- **Tầng ưu tiên cao:** Kích hoạt truy vấn trực tiếp từ chip giải mã GPS phần cứng của thiết bị với thời gian giới hạn chờ là 6 giây nhằm đạt độ chính xác tối đa.
- **Tầng dự phòng:** Tự động chuyển đổi sang phương thức định vị dựa trên trạm phát WiFi, địa chỉ IP hoặc định vị trạm sóng di động với thời gian chờ 10 giây trong trường hợp phần cứng GPS không phản hồi kịp thời.

```

getCurrentPosition() {
  if (!navigator.geolocation) {
    return Promise.reject(new Error('Trình duyệt không hỗ trợ định vị GPS'));
  }

  const tryGPS = (highAccuracy, timeout) =>
    new Promise((resolve, reject) => {
      navigator.geolocation.getCurrentPosition(
        pos => resolve({ lat: pos.coords.latitude, lng: pos.coords.longitude }),
        err => reject(err),
        { enableHighAccuracy: highAccuracy, timeout: timeout, maximumAge: 30000 }
      );
    });

  // Tầng 1: high-accuracy (GPS chip) - nhanh, timeout 6 s
  return tryGPS(true, 6000).catch(() =>
    // Tầng 2: low-accuracy (IP/WiFi/cell) - timeout 10 s
    tryGPS(false, 10000).catch(() => {
      throw new Error(
        'Không lấy được vị trí GPS. Hãy dùng "Chọn điểm bắt đầu trên bản đồ" để chỉ định vị trí
      );
    })
  );
},

```

Hình 3.9 Phương thức lấy tọa độ vị trí hiện tại của người dùng

- Trong kịch bản cả hai phương thức đều thất bại, hệ thống sẽ hiển thị thông báo hướng dẫn người dùng chủ động chỉ định vị trí xuất phát thủ công bằng cách tương tác trực tiếp trên giao diện bản đồ.

Phương thức findNearestNode(lng, lat): Kích hoạt yêu cầu truy vấn đến điểm cuối GET /api/paths/nodes?near=lng,lat nhằm tìm kiếm mã định danh của nút giao thông thuộc mạng lưới đường nội khu có khoảng cách gần nhất với vị trí hiện thời của người dùng.

```

async findNearestNode(lng, lat) {
  const res = await fetch(`${API_BASE}/paths/nodes?near=${lng},${lat}`);
  if (!res.ok) throw new Error('Không thể tìm node gần nhất');
  const result = await res.json();
  if (!result.data) throw new Error('Không có node nào trong DB');
  return result.data;
},

```

Hình 3.10 Thuật toán tìm road_node gần nhất với tọa độ (lng, lat)

Phương thức findBestGate(destLng, destLat): Hệ thống thiết lập sẵn danh mục tọa độ không gian của 4 cổng chính dẫn vào khuôn viên trường. Thuật toán tiến hành sắp xếp thứ hạng các cổng dựa trên khoảng cách Euclid tính toán đến điểm đích không gian của người dùng, từ đó lựa chọn cổng có khoảng cách ngắn nhất nhằm mục đích tối thiểu hóa chi phí tính toán của giải thuật A* bên trong nội khu. Mã số định danh

của nút giao thông tại cổng được xử lý theo cơ chế hệ thống chỉ kích hoạt truy vấn API khi phát sinh nhu cầu thực tế lần đầu tiên, sau đó lưu trữ vào bộ nhớ đệm để tránh lặp lại các yêu cầu truy vấn trùng lặp.

```
async findBestGate(destLng, destLat) {
  // Tính khoảng cách từ mỗi cổng đến điểm đích
  const ranked = CAMPUS_GATES.map(gate => ({
    ...gate,
    dist: euclideanDist(gate.lng, gate.lat, destLng, destLat),
  })).sort((a, b) => a.dist - b.dist);

  const bestGate = ranked[0]; // Cổng gần đích nhất
```

Hình 3.11 Thuật toán xác định và lựa chọn cổng tiếp cận khuôn viên trường

3.3.3.2. Module tìm đường OSRM

Trong kịch bản hệ thống xác định vị trí của người dùng nằm ngoài ranh giới không gian của trường, ứng dụng yêu cầu thiết lập một tuyến đường nối liền từ vị trí hiện hành đến vị trí cổng tiếp cận nội khu đã chọn. Module osrm.js chịu trách nhiệm thiết lập kênh giao tiếp trực tiếp với dịch vụ định tuyến đường bộ công cộng OSRM thông qua địa chỉ máy chủ router.project-osrm.org. Kết quả phản hồi từ dịch vụ là một mảng tập hợp các tọa độ địa lý [lat, lng] sau khi trải qua bước giải mã từ chuỗi định dạng nén dữ liệu hình học (Polyline mã hóa) của OSRM, sẵn sàng chuyển tiếp đến thành phần PathRenderer để thực thi tiến trình vẽ đồ họa.

```
async getRoute(fromLng, fromLat, toLng, toLat) {
  const url = `${OSRM_BASE}/${fromLng},${fromLat};${toLng},${toLat}` +
    '?overview=full&geometries=geojson&steps=false';
  const res = await fetch(url);
  if (!res.ok) throw new Error('Không thể kết nối OSRM');

  const data = await res.json();

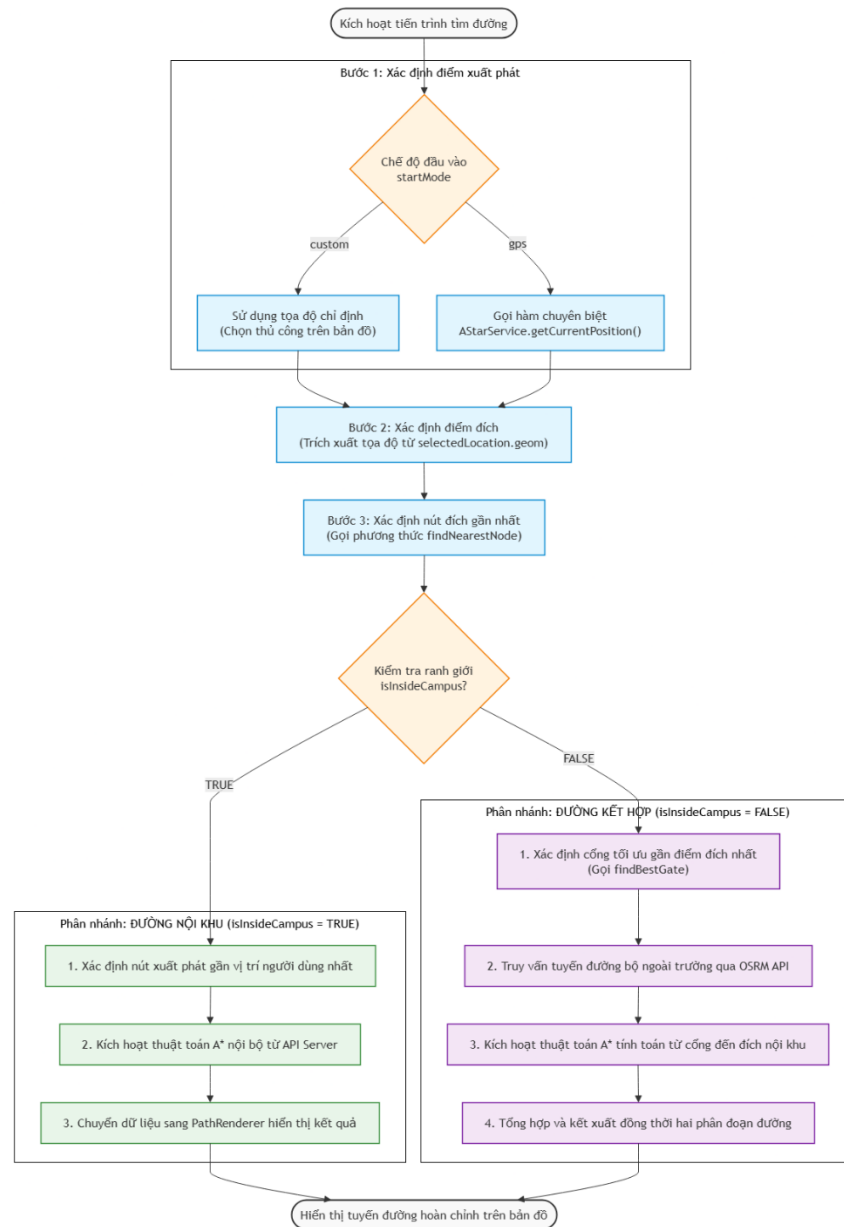
  if (data.code !== 'Ok' || !data.routes || data.routes.length === 0) {
    throw new Error('OSRM không tìm được tuyến đường');
  }

  const route = data.routes[0];
  // GeoJSON coords là [lng, lat] → chuyển sang [lat, lng] cho Leaflet
  const coords = route.geometry.coordinates.map(([lng, lat]) => [lat, lng]);

  return {
    coords, // Array<[lat, lng]> cho Leaflet polyline
    distanceMeters: route.distance, // Khoảng cách (m)
    durationSeconds: route.duration, // Thời gian ước tính (giây)
  };
};
```

Hình 3.12 Module tìm đường OSRM

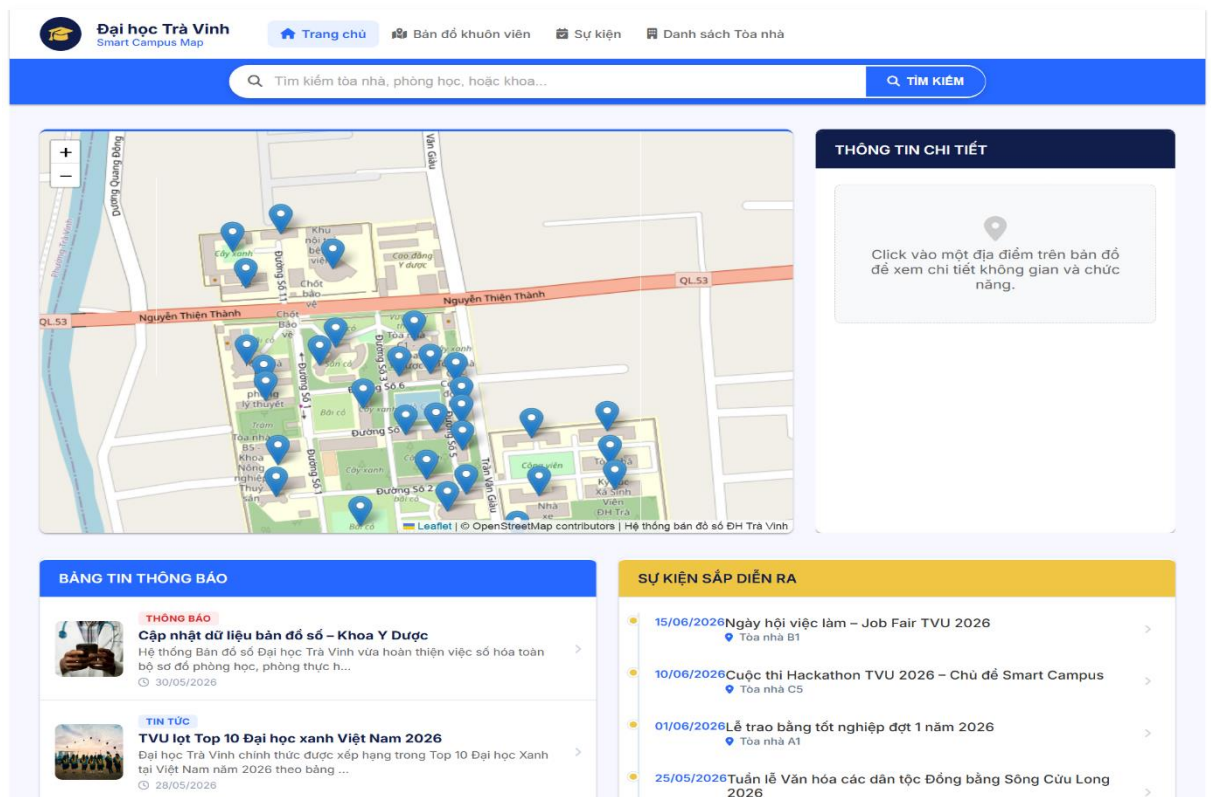
3.3.3.3. Luồng xử lý tìm đường



Hình 3.13 Sơ đồ luồng tiến trình xử lý tìm đường

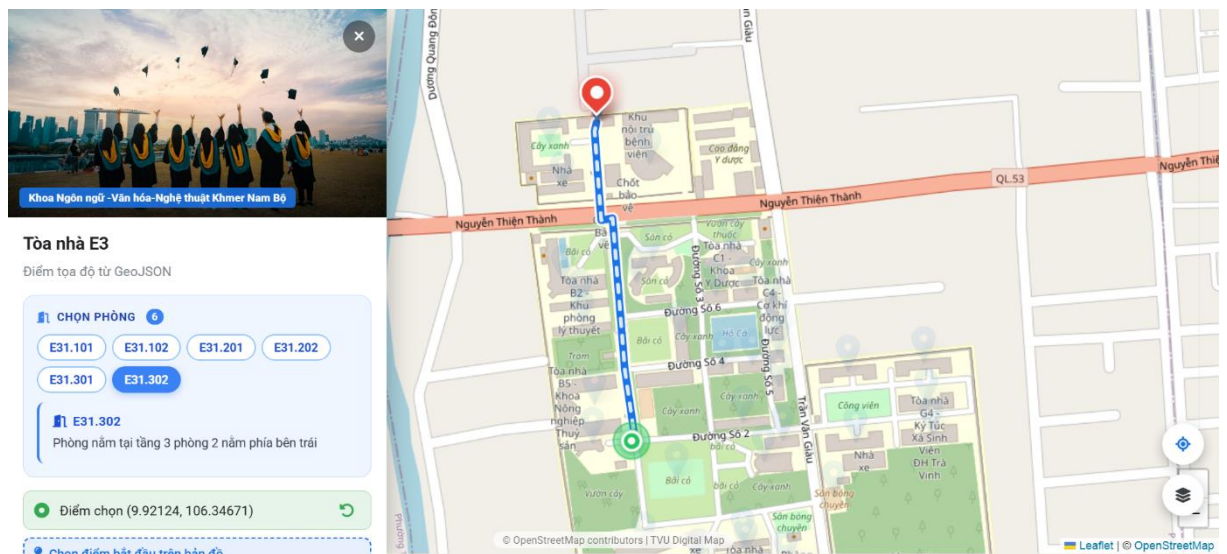
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU

4.1. Giao diện trang chủ

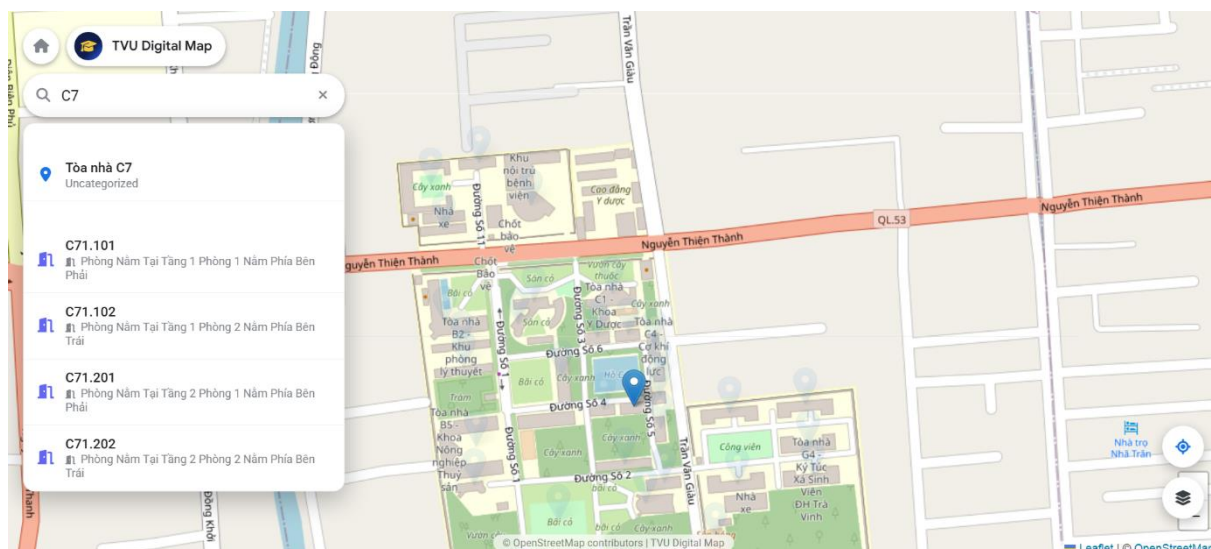


Hình 4.1 Giao diện trang chủ

4.2. Giao diện trang bản đồ

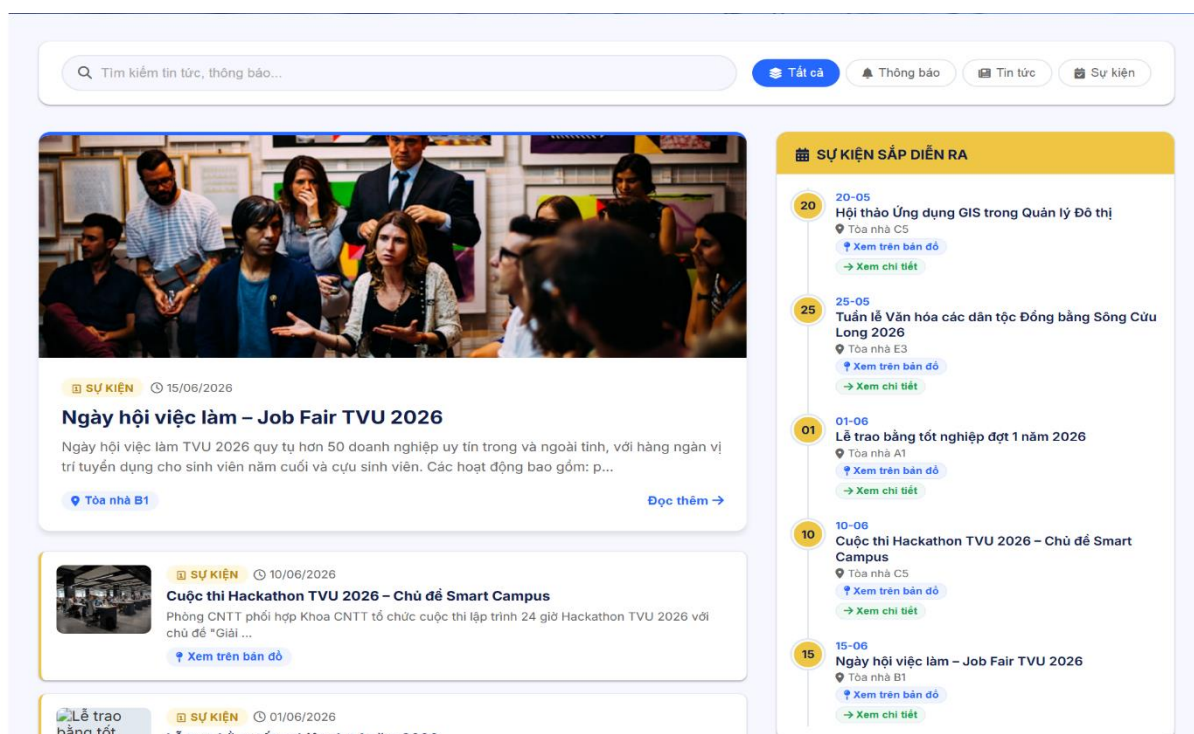


Hình 4.2 Giao diện trang bản đồ

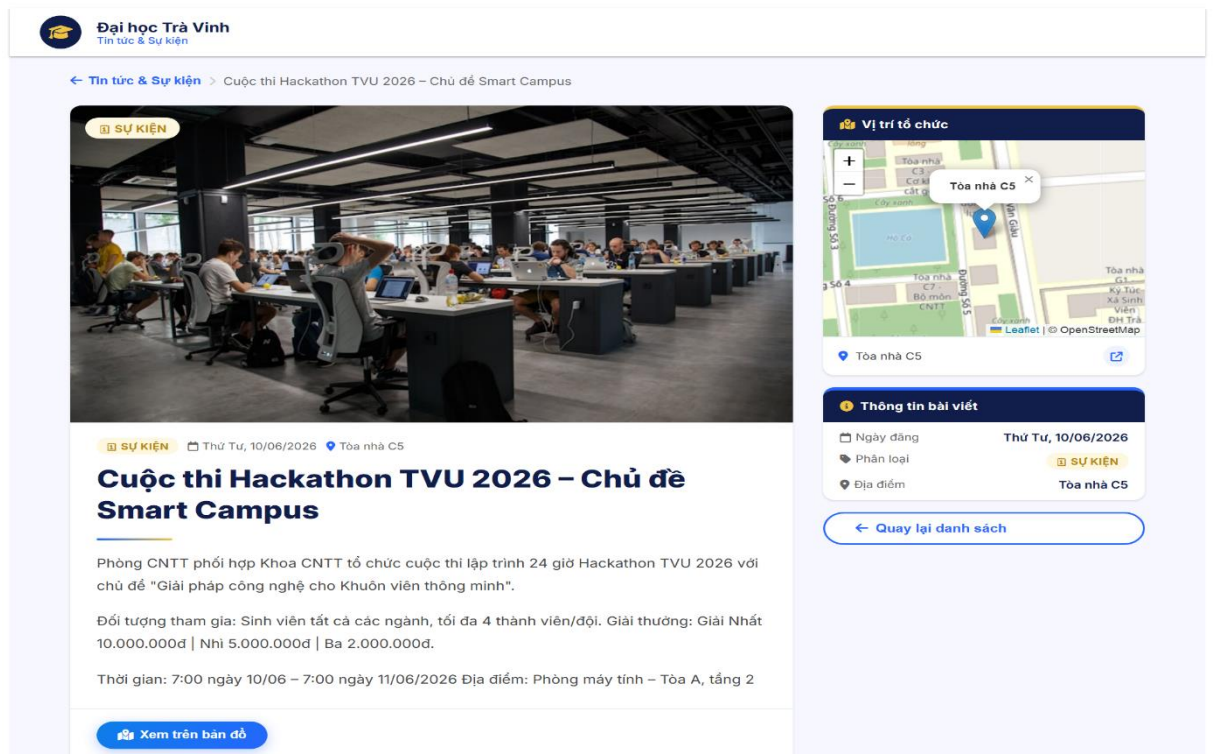


Hình 4.3 Giao diện chức năng tìm kiếm trong trang bản đồ

4.3. Giao diện trang tin tức

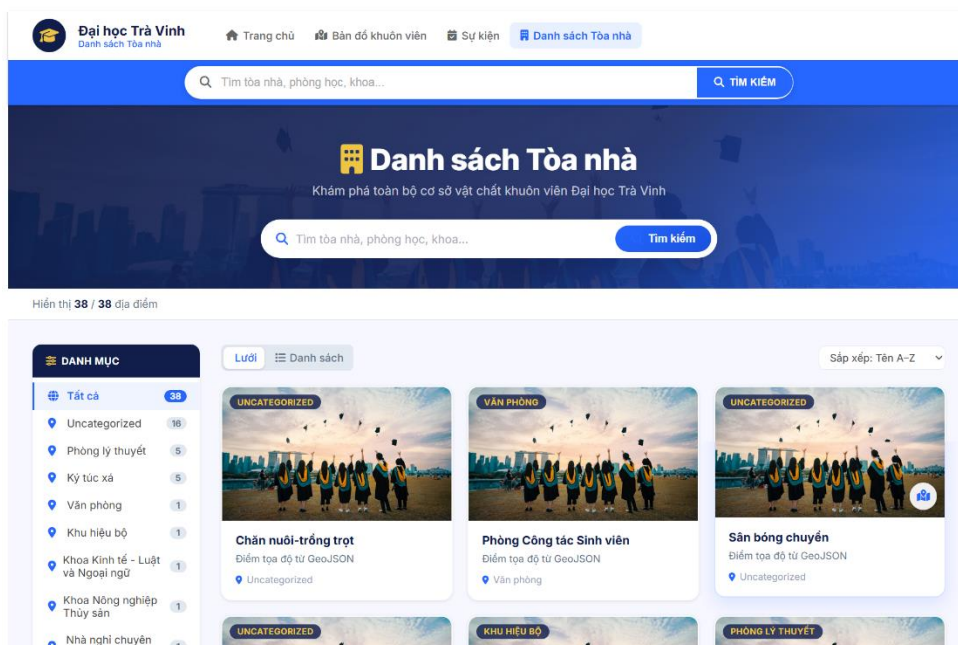


Hình 4.4 Giao diện trang tin tức



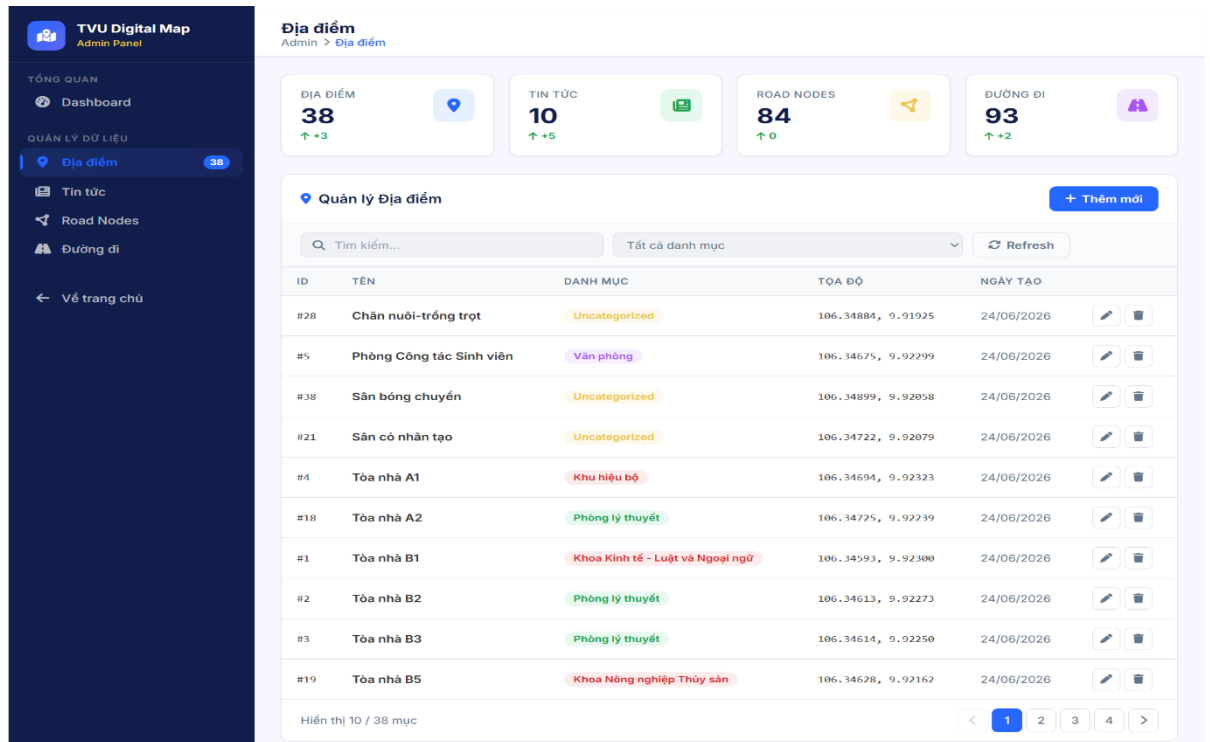
Hình 4.5 Giao diện trang đọc tin tức

4.4. Giao diện trang danh sách tòa nhà



Hình 4.6 Giao diện trang danh sách tòa nhà

4.5. Giao diện trang quản trị



Hình 4.7 Giao diện trang quản trị

CHƯƠNG 5: KẾT LUẬN VÀ PHƯƠNG HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong đồ án "Phát triển ứng dụng bản đồ số hỗ trợ sinh viên tìm phòng học trong khuôn viên Đại học Trà Vinh", tôi đã hoàn thành toàn bộ các mục tiêu đặt ra ban đầu và thu được những kết quả cụ thể như sau:

- Đã nghiên cứu và làm chủ được các công nghệ cốt lõi trong lĩnh vực thông tin địa lý (GIS) bao gồm việc quản lý dữ liệu không gian bằng hệ quản trị cơ sở dữ liệu PostgreSQL kết hợp phần mở rộng PostGIS; ứng dụng thư viện Leaflet trong việc hiển thị bản đồ và xử lý lớp dữ liệu GeoJSON; đồng thời tối ưu hóa thuật toán A* để áp dụng vào bài toán tìm đường đi ngắn nhất trong khuôn viên trường.
- Xây dựng hoàn chỉnh ứng dụng WebApp trên nền tảng Vue.js (Frontend) và Node.js (Backend) với giao diện trực quan, có tính tương thích cao và phản hồi mượt mà trên các thiết bị di động.
- Hiện thực hóa chức năng tìm kiếm nhanh phòng học, tòa nhà, các tiện ích (thư viện, nhà xe, căn tin) và chức năng chỉ đường tối ưu từ vị trí hiện tại đến phòng học đích thông qua thuật toán A*.

5.2. Hướng phát triển

Tối ưu hóa và nâng cấp thuật toán tìm đường: Nghiên cứu và cải tiến thuật toán A* kết hợp với dữ liệu độ cao để xây dựng tính năng chỉ đường 3D xuyên tầng.

Phát triển ứng dụng di động: Chuyển đổi hoặc phát triển thêm phiên bản Mobile App sử dụng các framework như Vue Native hoặc Capacitor để tận dụng tối đa phần cứng thiết bị (như la bàn số, cảm biến bước chân) nhằm nâng cao trải nghiệm chỉ đường thời gian thực.

Mở rộng tiện ích thông minh: Kết nối ứng dụng bản đồ với hệ thống quản lý lịch học của trường. Sinh viên chỉ cần đăng nhập là ứng dụng có thể tự động đồng bộ thời khóa biểu, nhắc lịch học và tự động vạch sẵn lộ trình di chuyển đến phòng học của buổi học ngày hôm đó.

DANH MỤC TÀI LIỆU THAM KHẢO

1. Brown, E. (2019). *Web Development with Node and Express: Leveraging the JavaScript Stack* (2nd edn). O'Reilly Media.
2. Delling, D., Sanders, P., Schultes, D., & Wagner, D. (2009). Engineering route planning algorithms. In *Algorithmics of Large and Complex Networks* (pp. 117–139). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-02094-0_7
3. Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. *ACM SIGMOD Record*, 14(2), 47–57. <https://doi.org/10.1145/971697.602266>
4. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>
5. Longley, P. A., Goodchild, M. F. carrot, Maguire, D. J., & Rhind, D. W. (2015). *Geographic Information Science and Systems* (4th edn). Wiley.
6. Obe, R., & Hsu, L. (2021). *PostGIS in Action* (3rd edn). Manning Publications.
7. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th edn). Pearson.
8. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6), 80–83. <https://doi.org/10.1109/MIC.2010.145>
9. Vue.js Core Team. (2023). *Vue 3 Official Documentation – Composition API FAQ*. <https://vuejs.org/guide/extras/composition-api-faq.html>
10. You, E. & Vue.js Contributors. (2022). *Vue 3 Migration Guide*. <https://v3-migration.vuejs.org/>